



SCHOOL OF BUILT ENVIRONMENT, ENGINEERING AND
COMPUTING

LEEDS BECKETT UNIVERSITY

Design and Implementation of a Near Real-Time Machine Learning Based Intrusion Detection System for Network Traffic Analysis

By: Mathias Amade

Supervisor: Mo Hassan

Submitted to Leeds Beckett University in partial fulfilment of the requirements for
the degree of MSc Cyber Security

September 2026

Candidate's Declaration

I, Mathias Amade, confirm that this dissertation and the work presented in it are my own achievement.

Where I have consulted the published work of others this is always clearly attributed;

Where I have quoted from the work of others the source is always given. With the exception of such quotations this dissertation is entirely my own work;

I have acknowledged all main sources of help;

I have read and understand the penalties associated with Academic Misconduct.

Signed: Mathias Amade

Date: 7th September 2026

Student ID No:

Acknowledgements

I would first like to thank God for His grace, strength and guidance throughout my MSc programme and the completion of this dissertation.

I am sincerely grateful to my supervisor, Mo Hassan, for his guidance, constructive feedback and encouragement throughout this project. His advice helped me refine the scope of the research and improve the quality of the final work.

I would also like to thank the lecturers and staff of the School of Built Environment, Engineering and Computing at Leeds Beckett University for the knowledge and support provided throughout my studies.

My deepest appreciation goes to my family for their prayers, sacrifices, encouragement and constant support throughout this journey. Their belief in me gave me the strength to continue, especially during the most demanding stages of the programme.

I am also grateful to my close friends for their patience, encouragement, regular check-ins and moral support. Their presence and reassurance made the journey easier and reminded me that I was not alone.

Finally, I appreciate everyone who contributed in one way or another to the successful completion of this dissertation.

Abstract

Machine learning has shown strong potential for network intrusion detection, but many existing studies focus mainly on offline classification performance without considering whether models can respond quickly enough for near real-time use. This research designed, implemented and evaluated a machine learning-based intrusion detection system under controlled near real-time conditions.

The UNSW-NB15 dataset was used to train and evaluate three supervised machine learning algorithms: Random Forest, Gradient Boosting and Support Vector Machine. The models were compared using accuracy, precision, recall, F1-score, ROC AUC, training time and prediction time. Random Forest was subsequently evaluated within a two-virtual-machine environment, where network-flow records were transmitted individually from a traffic generator to the intrusion detection system. Detector-side latency and streamed classification accuracy were measured during this experiment.

Gradient Boosting achieved the highest offline accuracy at 89.86%, followed closely by Random Forest at 89.75%, while Support Vector Machine achieved 87.93%. Despite the small differences in classification performance, the models differed considerably in computational efficiency. Random Forest provided the strongest overall balance between classification performance and computational cost and was therefore selected for the networked evaluation. During this experiment, it achieved 88.40% classification accuracy with a median detector-side latency of 26.36 milliseconds.

The findings demonstrate that classification accuracy alone is insufficient when assessing machine learning models for near real-time intrusion detection. Computational efficiency and response latency must also be considered. Although the prototype demonstrates near real-time feasibility under controlled conditions, further evaluation using live packet capture, real-time feature extraction and operational network traffic is required before production deployment.

Table of Contents

CANDIDATE’S DECLARATION	II
ACKNOWLEDGEMENTS	III
ABSTRACT.....	4
TABLE OF CONTENTS	5
LIST OF FIGURES.....	10
LIST OF TABLES	11
ABBREVIATIONS.....	11
CHAPTER 1: INTRODUCTION.....	12
1.1 OVERVIEW	12
1.2 RATIONALE	13
1.3 AIM AND OBJECTIVES	13
1.4 DISSERTATION STRUCTURE	14
CHAPTER 2: LITERATURE REVIEW.....	16
2.1 LITERATURE REVIEW METHODOLOGY	16
2.1.1 SEARCH KEYWORDS.....	16
2.1.2 INCLUSION CRITERIA	16
2.1.3 EXCLUSION CRITERIA	17
2.1.4 SELECTION PROCESS.....	17
2.2 OVERVIEW	17
2.3 MACHINE LEARNING FOR INTRUSION DETECTION.....	18
2.4 NETWORK TRAFFIC DATASETS.....	19

2.5 REAL-TIME DETECTION CHALLENGES	20
2.6 RESEARCH GAPS AND MOTIVATION	21
2.7 SUMMARY OF KEY LITERATURE	22
CHAPTER 3: METHODOLOGY	23
3.1 RESEARCH DESIGN AND APPROACH	24
3.2 INVESTIGATION SCOPE	25
3.2.1 SUPERVISED MACHINE LEARNING ONLY	26
3.2.2 ALGORITHM SELECTION AND RATIONALE	26
3.2.3 DATASET SCOPE	26
3.2.4 SCOPE OF THE PROJECT	27
3.2.5 PERFORMANCE EVALUATION SCOPE	27
3.2.6 SUMMARY OF RESEARCH SCOPE	27
3.3 DATA COLLECTION AND ANALYSIS	28
3.4 EVALUATION	29
3.5 ETHICAL ISSUES	30
CHAPTER 4: RESEARCH DESIGN AND IMPLEMENTATION	31
4.1 IMPLEMENTATION OVERVIEW	31
4.1.1 <i>Technology Stack</i>	31
4.2 DEVELOPMENT ENVIRONMENT SETUP	32
4.2.1 <i>Virtual Machine Architecture</i>	32
4.2.2 <i>Network Configuration</i>	32
4.2.3 <i>Software Installation and Configuration</i>	33
4.2.4 <i>Assessment of Environment Setup</i>	34
4.3 IMPLEMENTATION OF THE PREPROCESSING STEP	34

4.3.1 Data Loading and Initial Exploration	34
4.4 TESTING ENVIRONMENT ARCHITECTURE	36
4.5 DATA PREPROCESSING	37
4.5.1 Preprocessing Pipeline Implementation.....	37
4.5.2 Preprocessing Results	38
4.5.3 Critical Reflection on the Preprocessing.....	40
4.6 FEATURE SELECTION.....	40
4.6.1 Rationale and Methodology.....	41
4.6.2 Implementation.....	41
4.7 MACHINE LEARNING MODEL IMPLEMENTATION	45
4.7.1 Random Forest Implementation	45
4.7.2 Gradient Boosting Implementation	48
4.7.3 Support Vector Machine Implementation.....	51
4.7.4 Comparative Analysis of Machine Learning Models	54
4.8 TWO-VIRTUAL-MACHINE REAL-TIME DETECTION EVALUATION	56
4.8.1 Architecture and Setup.....	56
4.8.2 Real-Time Latency Results	57
4.8.3 Throughput and Detection Accuracy	59
CHAPTER 5: RESEARCH OUTCOMES, DISCUSSION AND EVALUATION	59
5.1 OVERVIEW OF KEY FINDINGS	60
5.2 COMPARATIVE PERFORMANCE WITH THE LITERATURE	61
5.3 THE ACCURACY VERSUS EFFICIENCY TRADE-OFF.....	62
5.4 NEAR REAL-TIME DETECTION FEASIBILITY	63
5.5 ACHIEVEMENT OF RESEARCH OBJECTIVES	63

5.6 LIMITATIONS OF THE RESEARCH	65
5.7 IMPLICATIONS AND FUTURE WORK	66
5.8 SUMMARY	68
CHAPTER 6: PROJECT MANAGEMENT	68
6.1 PROJECT PLANNING AND SCHEDULING	68
6.2 PROJECT EXECUTION AND TIME MANAGEMENT	69
6.3 SUPERVISOR ENGAGEMENT AND FEEDBACK	70
6.4 CHALLENGES AND SCOPE CHANGES	71
6.5 REFLECTION AND LESSONS LEARNED	72
6.6 SUMMARY	73
CHAPTER 7: CONCLUSION AND FUTURE WORK	74
7.1 SUMMARY OF THE RESEARCH	74
7.2 KEY CONTRIBUTIONS.....	74
7.3 ACHIEVEMENT OF AIM AND OBJECTIVES	75
7.4 CRITICAL EVALUATION	76
7.5 FUTURE WORK.....	77
7.6 CONCLUDING REMARKS.....	77
REFERENCES	79
APPENDICES	80
APPENDIX A	80
ORIGINALITY AND USE OF GENERATIVE ARTIFICIAL INTELLIGENCE (GAI) STATEMENT	80
APPENDIX B	83
ETHICS APPROVAL	83

APPENDIX C: EVIDENCE OF MEETINGS WITH SUPERVISORS	84
APPENDIX D: IMPLEMENTATION CODE	85
D.0 ENVIRONMENT SETUP COMMANDS	86
D.1 DATA PREPROCESSING (`02_PREPROCESS_DATA.PY`).....	87
D.2 FEATURE SELECTION (`03_FEATURE_SELECTION.PY`).....	89
D.3 RANDOM FOREST TRAINING (`04_TRAIN_RANDOM_FOREST.PY`).....	92
D.4 IDS DETECTOR — VM2 (`07_IDS_DETECTOR.PY`)	94
D.5 TRAFFIC GENERATOR — VM1 (`08_TRAFFIC_GENERATOR.PY`)	97
D.6 LATENCY VISUALISATION (`09_LATENCY_CHART.PY`)	99

List of Figures

Figure 2.1 Machine Learning-Based Intrusion Detection System Workflow

Figure 2.2 UNSW-NB15 Dataset Structure and Attack Categories

Figure 3.1 Proposed ML-Based IDS System Architecture

Figure 3.2 Two-VM Testing Environment

Figure 4.1 Two-Virtual-Machine Network Architecture

Figure 4.2 UNSW-NB15 Dataset Exploration Output

Figure 4.3 Preprocessing Pipeline Output Results

Figure 4.4 Feature Selection Script Output

Figure 4.5 Top 20 Feature Importance Scores (Random Forest)

Figure 4.6 Random Forest Training and Evaluation Terminal Output

Figure 4.7 Random Forest Confusion Matrix

Figure 4.8 Gradient Boosting Training and Evaluation Script Output

Figure 4.9 Gradient Boosting Confusion Matrix

Figure 4.10 Support Vector Machine Training and Evaluation Script Output

Figure 4.11 Support Vector Machine Confusion Matrix

Figure 4.12 Two-VM Real-Time Detection Experiment Output (VM2 Detector)

Figure 4.13 Distribution of Detector-Side Processing Latency for 990 Flows

List of Tables

Table 2.1 Summary of Key Literature Reviewed

Table 4.1 Preprocessing Pipeline Summary

Table 4.2 Top 10 Most Important Features

Table 4.3 Random Forest Performance Metrics on UNSW-NB15 Test Set

Table 4.4 Gradient Boosting Performance Metrics on UNSW-NB15 Test Set

Table 4.5 Support Vector Machine Performance Metrics on UNSW-NB15 Test Set

Table 4.6 Comparative Performance of Random Forest, Gradient Boosting, and SVM

Table 4.7: Two-VM Detector-side Latency Statistics (990 flows, Random Forest)

Table 5.1 Consolidated Summary of Key Results

Abbreviations

AUC: Area Under the Curve

DoS: Denial of Service

DSR: Design Science Research

GB: Gradient Boosting

IDS: Intrusion Detection System

ML: Machine Learning

RF: Random Forest

ROC: Receiver Operating Characteristic

SVM: Support Vector Machine

VM: Virtual Machine

Chapter 1: Introduction

1.1 Overview

Network intrusion detection systems are essential for detecting malicious activity in today's computer networks. Organisations need security solutions that identify suspicious traffic that could escape firewalls and other security systems.

The applications of machine learning in intrusion detection are already widespread as it is capable of detecting any pattern in network traffic and recognizing the normal or malicious activity. The performance in classification is excellent in recent studies using benchmark datasets. Most of the existing studies, however, are offline studies and the performance of these models in the case where the network traffic needs to be processed continuously and without significant delay is less studied (Imanbayev et al., 2022).

In this project, supervised machine learning for real-time network intrusion detection is explored. The proposed system is trained and tested with the UNSW-NB15 dataset that consists of normal traffic and various modern attack types: exploits, denial-of-service attacks, reconnaissance, backdoors and fuzzers (Moustafa and Slay, 2015).

The supervised learning algorithms that are tested are: Random Forest, Gradient Boosting and Support Vector Machine. The performance of these models is evaluated using classification metrics: accuracy, precision, recall, F1-score and confusion matrix. They are also evaluated with regard to their computational performance, as measured by the time needed for training and for prediction and response-latency.

The system is tested in a virtual-machine environment of two virtual machines in order to support the near real-time evaluation. Both machines are virtual machines: one is the traffic generator and the other is the intrusion detection system and is used for classification. This controlled setup allows the study to both check the detection performance and the time it takes to process the incoming traffic.

This study examines whether the selected machine learning models can detect intrusions accurately while maintaining a low response time under the experimental conditions used in this project.

1.2 Rationale

Previous studies have reported promising classification results for machine learning-based intrusion detection systems. However, many of these evaluations were conducted offline using benchmark datasets. Fewer studies have investigated whether the models can process incoming network traffic quickly enough to support near real-time intrusion detection (Imanbayev et al., 2022).

This is important because an effective intrusion detection system must do more than classify traffic accurately. Suspicious activity must also be identified early enough to support further investigation and possible containment. When detection is delayed, an attacker may have more time to remain within the network and cause additional damage.

For this reason, the project examines whether supervised machine learning models can distinguish between normal and malicious network traffic while maintaining a suitable response time. Random Forest, Gradient Boosting and Support Vector Machine are compared using the UNSW-NB15 dataset under controlled laboratory conditions. Their performance is evaluated using both classification results and detection time.

The study is also motivated by the practical need for intrusion detection systems that can identify malicious activity without unnecessary delay. As cyber threats continue to affect organisations, detection speed must be considered alongside classification accuracy. An intrusion detection system that detects attacks too slowly may provide limited practical value, regardless of its predictive performance (Imanbayev et al., 2022).

Consequently, this study evaluates supervised machine learning models using both traditional classification metrics and response latency to provide a more realistic assessment of their suitability for near real-time intrusion detection.

1.3 Aim and Objectives

The aim of this study is to design, implement and evaluate a machine learning-based intrusion detection system capable of detecting malicious network traffic under controlled near real-time conditions.

The objectives are:

1. To critically review the literature on traditional and machine learning-based intrusion detection systems.
2. To develop a machine learning-based intrusion detection system using Python and the UNSW-NB15 dataset.
3. To train and evaluate Random Forest, Gradient Boosting and Support Vector Machine models for binary network traffic classification.
4. To compare the models using accuracy, precision, recall, F1-score, confusion matrices, training time and prediction time, and to evaluate detector-side response latency for the selected model.
5. To assess the feasibility of the proposed system within a controlled two-virtual-machine environment.

1.4 Dissertation Structure

This dissertation is organised as follows:

The research is introduced in chapter 1, which gives an overview of the research, the rationale for the research and an outline of project aims and objectives.

In Chapter 2 a detailed literature study is conducted of relevant literature and is followed by a discussion about the rule-based approach to intrusion detection, machine learning based intrusion detection systems, real time network intrusion detection systems, and the gap in literature that this dissertation seeks to address.

The methodology for the study is explained in Chapter 3: Research Methodology, which covers the research design, scope and boundaries of the study, the UNSW-NB15 dataset, the machine learning algorithms used in the study, the architecture for the two-virtual-machine testing environment, description of evaluation metrics and ethical considerations.

Chapter 4 presents the implementation of the proposed intrusion detection system, including the development environment, dataset preprocessing, model training and the two-virtual-machine testing environment.

The results of the research conducted for each machine learning algorithm (detection accuracy), the latency, and the discussion of the research findings in relation to the project aims and literature review are detailed and discussed in Chapter 5.

Project planning, execution and risk management, as well as lessons learned from collaborating with the project's supervisor and managing the project timeline are discussed in chapter 6.

The dissertation is completed by a chapter at the end, which is used to summarise the main findings, to assess the successful implementation of the project aims and objectives, to identify limitations and to propose lines of further research.

Chapter 2: Literature Review

A review of the previous work on intrusion detection systems is presented in this chapter. Emphasis is given to machine learning techniques and the problems that are encountered in attempting to get near real-time detection. The methods used by intrusion detection systems are explained and the areas of this research that motivates the dissertation.

2.1 Literature Review Methodology

The literature review followed a structured process to identify and assess studies on machine learning-based intrusion detection systems. Relevant papers were identified through defined search terms and then screened using the inclusion and exclusion criteria. The selected studies were reviewed to understand existing approaches, compare their findings and identify the research gap addressed in this project.

Literature was retrieved from IEEE Xplore, ScienceDirect, ACM Digital Library, SpringerLink and Google Scholar to ensure broad coverage of cybersecurity, networking and machine learning research. Additional searches included the UNSW research repository and selected industry publications where relevant to intrusion detection and contemporary cybersecurity practices.

2.1.1 Search Keywords

Searches combined terms including intrusion detection system, machine learning, network security, anomaly detection, Random Forest, Gradient Boosting, Support Vector Machine, UNSW-NB15 and real-time detection. Boolean operators such as AND, OR, NOT were used to refine the results and improve relevance.

2.1.2 Inclusion Criteria

Studies were included where they:

- were published between 2018 and 2026, with earlier foundational studies retained where relevant;
- were peer-reviewed journal articles or reputable conference papers;
- were written in English;
- focused on machine learning-based network intrusion detection;
- included empirical evaluation or comparative analysis; and

- were relevant to network traffic classification, anomaly detection or near real-time intrusion detection.

Foundational studies published before 2018 were included selectively where they made a significant contribution to the field, particularly Moustafa and Slay (2015), which introduced the UNSW-NB15 dataset.

2.1.3 Exclusion Criteria

Studies were excluded where they:

- focused exclusively on host-based intrusion detection rather than network-based detection;
- addressed only narrowly defined application-layer attacks without broader relevance to network intrusion detection;
- lacked empirical evaluation or comparative analysis;
- were opinion articles, non-peer-reviewed publications or duplicated findings already covered by more recent studies; or
- were not sufficiently relevant to the research aim.

Industry reports were retained only where they provided current cybersecurity context from established organisations.

2.1.4 Selection Process

The database searches produced a range of publications relating to machine learning-based intrusion detection. After the inclusion and exclusion criteria were applied and duplicate or irrelevant results were removed, the most relevant studies were retained for detailed review.

The selected literature was organised into four themes: machine learning approaches for intrusion detection, network traffic datasets and benchmarking, near real-time detection challenges, and comparative evaluations of machine learning algorithms. Using this process ensured that only studies relevant to the research objectives were included in the review.

2.2 Overview

Enterprise networks are becoming increasingly complex and connected, making malicious network activity harder to detect using traditional security controls such as firewalls. This is especially the case when attackers bypass perimeter defences and

move laterally across a network (Yeoh et al., 2023). The primary role of an intrusion detection system is to detect suspicious activity within an organisation's network, particularly when malicious traffic bypasses perimeter security controls.

This chapter focuses on three areas of interest to this project. Firstly, the use of machine learning for the area of intrusion detection. Secondly, the common datasets that are used in the training and evaluation of intrusion detection system models. Finally, it examines the challenges associated with adapting these techniques for near real-time intrusion detection.

2.3 Machine Learning for Intrusion Detection

Machine learning has become a widely adopted approach for intrusion detection because it can learn patterns from historical network traffic and apply them to classify new traffic. Unlike signature-based methods, machine learning may identify previously unseen attack patterns when sufficient representative training data are available.

Several supervised learning algorithms have been successfully applied to intrusion detection. Random Forest, Gradient Boosting and Support Vector Machine (SVM) are among the most widely studied approaches and have demonstrated strong classification performance on benchmark datasets (Alhakeem and Bin Ajan, 2024). However, high classification accuracy does not automatically translate into effective operational performance.

Random Forest is widely used because it is effective for high-dimensional data and is generally less susceptible to overfitting than an individual decision tree. Random Forest classifiers have been widely used in intrusion detection research and have produced strong classification results across different benchmark datasets (Ahmad and AlQahtani, 2023).

Gradient Boosting has also been applied in network intrusion detection. It builds decision trees in sequence, with each new tree learning from the errors made by the previous ones. This approach has shown promising results in network traffic classification (Ahmad and AlQahtani, 2023).

Support Vector Machine uses a different method by finding a decision boundary that separates classes within the feature space. SVM has been used to separate normal

and malicious network traffic, including datasets with complex feature relationships between features (Alhakeem and Bin Ajlan, 2024).

Deep learning approaches, such as CNN-LSTM, have also been investigated for intrusion detection. Although they can achieve strong classification performance, they are often more complex and require greater computational resources. This may increase processing time, so simpler machine learning models may be more suitable where detection speed is an important requirement (Halbouni et al., 2022).

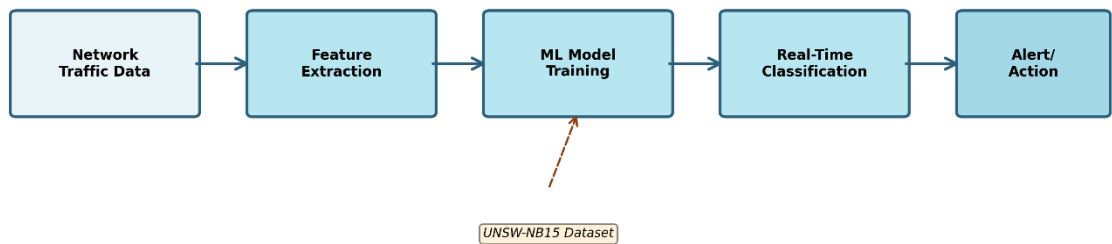


Figure 2.1: Machine Learning-Based Intrusion Detection System Workflow

Figure 2.1 shows the main stages of a machine learning-based intrusion detection system. These stages include collecting and preprocessing network traffic, training the model and using it to classify traffic. A similar process was used in this study, with detection time also measured during the experimental evaluation.

2.4 Network Traffic Datasets

The dataset used in intrusion detection research can affect how well a model performs because datasets differ in their quality, features and traffic patterns. This project chose UNSW-NB15 as its dataset because it contains both normal traffic and attacks with a variety of up-to-date attack categories such as exploits, shellcode, fuzzers, DoS attacks, and reconnaissance (Moustafa and Slay, 2015).

UNSW-NB15 contains more than 2.5 million network records and 49 traffic-related features. These features encompass multiple dimensions of network behaviour and provide ample information for training machine learning models to differentiate normal from malicious traffic. In contrast to legacy datasets such as KDD Cup 99, UNSW-NB15 mirrors a more contemporary network environment and includes a broader range

of attack categories, making it more suitable for evaluating intrusion detection systems (Moustafa and Slay, 2015).

The large number of records and diverse attack categories make the dataset suitable for intrusion detection research and allow meaningful comparison with previous studies. UNSW-NB15 has been widely adopted within the intrusion detection community and provides outcomes comparable to previous studies. Nonetheless, several constraints must be considered, specifically that the dataset was created in a laboratory environment and does not directly reflect traffic from operating networks. Furthermore, the ratio of normal to attack traffic is more balanced than what is typically seen within a live network, given that malicious traffic is generally a smaller component of the overall traffic.

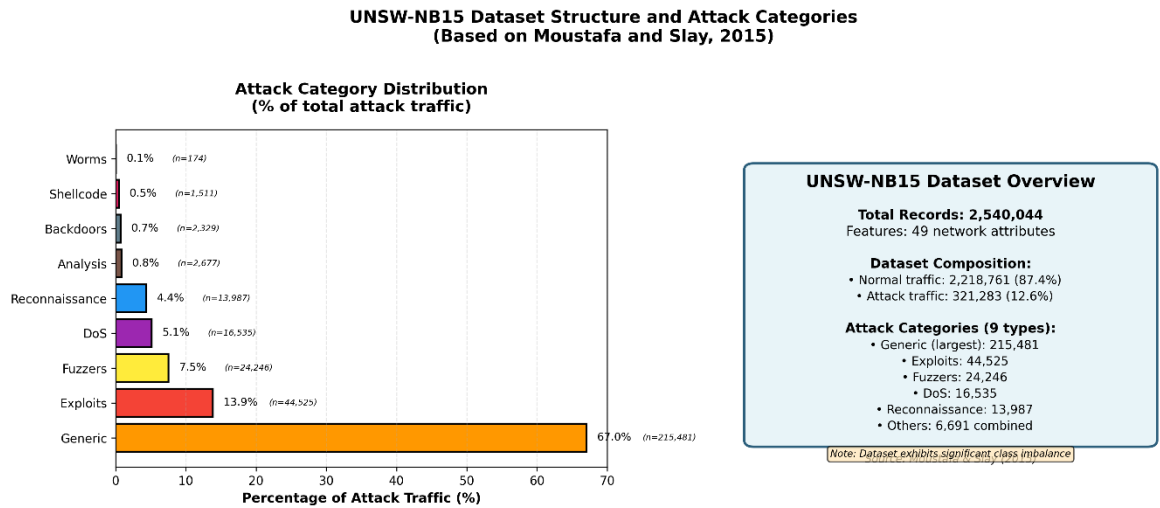


Figure 2.2: UNSW-NB15 Dataset Structure and Attack Categories

Source: Adapted from Moustafa and Slay (2015).

Figure 2.2 shows the structure of the UNSW-NB15 dataset with the data format and the type of information stored in each field. The dataset contains both normal and malicious traffic records, making it suitable for training and evaluating machine learning models for intrusion detection (Moustafa and Slay, 2015).

2.5 Real-time detection challenges

Intrusion detection studies often assess machine learning models based on offline benchmark datasets, reporting classification accuracies on previously unseen testing data. While the model may be able to correctly classify these test records, it does not

guarantee that the system can accurately and continuously process real network traffic in an operational environment (Imanbayev et al., 2022).

A model that achieves good classification accuracy offline may still be unsuitable for real-time intrusion detection, if the model takes too long to preprocess incoming traffic and classify it. Detecting threats with too much delay may minimise the chances of security experts to investigate suspicious traffic and contain an attack before further damage occurs (Imanbayev et al., 2022).

Common evaluation metrics such as precision, recall and F1-score offer useful insights into how well a model can classify a given set of traffic; however, these do not provide direct information on the latency a model experiences when presented with new traffic. Consequently, additional performance measures such as preprocessing time, model prediction time and model response latency should be considered when evaluating near real-time intrusion detection systems. In addition, the model-loading time and the model's computational-resource requirements for attack detection, along with the maximum volume of traffic that can be processed successfully per second, or minute are equally important practical considerations for near real-time intrusion detection (Imanbayev et al., 2022).

Accordingly, this study evaluates response latency alongside classification performance to assess the suitability of supervised machine learning models for near real-time intrusion detection within a controlled two-virtual-machine environment.

2.6 Research Gaps and Motivation

Recent studies have demonstrated that machine learning can provide efficient support for network intrusion detection systems (Alhakeem and Bin Ajlan, 2024; Imanbayev et al., 2022). However, most studies have focused on classification accuracy using offline benchmark datasets, with comparatively little attention given to response latency under near real-time conditions (Imanbayev et al., 2022).

Despite recent progress in the application of supervised learning in the fields of intrusion detection, machine learning, and cybersecurity, significant problems remain, and these issues are central to the functionality and usefulness of an intrusion detection system. The main gap identified in the literature is that few studies evaluate supervised machine learning models using both classification performance and detector-side response latency within the same experiment. High classification

accuracy alone is insufficient if preprocessing and prediction introduce delays that reduce operational effectiveness in near real-time environments. Accordingly, this study evaluates Random Forest, Gradient Boosting, and Support Vector Machine using a common preprocessing pipeline, the UNSW-NB15 dataset and a controlled two-virtual-machine environment. The models are compared using accuracy, precision, recall, F1-score, confusion matrices, training time, prediction time, and detector-side response latency.

2.7 Summary of Key Literature

Table 2.1 summarizes the major findings of the studies included in this chapter. It highlights the contributions of each study, the limitations and the research gaps most pertinent to machine learning-based intrusion detection.

Author(s) and Year	Key Contributions	Limitations/Gaps
Moustafa and Slay (2015)	Introduced the UNSW-NB15 dataset containing 2.5 million records and 49 features covering modern attack types including exploits, reconnaissance, DoS, backdoors, and fuzzers.	Dataset generated in controlled environment. Class balance artificially adjusted. May not fully reflect operational network traffic.
Ahmad and AlQahtani (2023)	Compared Random Forest, Gradient Boosting and a deep neural network for intrusion detection. Reported strong classification performance and identified Random Forest as the strongest model across several metrics.	Focused mainly on offline classification performance and did not evaluate detector-side latency within a networked environment.
Alhakeem and Bin Ajan (2024)	Comprehensive review of supervised machine	Review paper only with no empirical implementation.

	learning approaches for network anomaly detection. Highlighted generalisation challenges. Discussed feature selection importance.	Limited discussion of computational efficiency and deployment considerations.
Imanbayev et al. (2022)	Highlighted the gap between offline accuracy metrics and real-time deployment performance. Discussed feature extraction latency issues.	Proposed framework was not empirically validated. Limited comparison of specific algorithms under time constraints.
Halbouni et al. (2022)	Developed a hybrid CNN-LSTM model for network intrusion detection and reported strong classification performance.	Used a more complex deep-learning architecture and focused mainly on offline evaluation rather than detector-side latency.

Table 2.1: Summary of Key Literature Reviewed

The reviewed literature demonstrates that supervised machine learning techniques have consistently achieved strong classification performance for network intrusion detection. However, most existing studies have prioritised offline evaluation using benchmark datasets, with comparatively limited attention given to response latency and near real-time deployment. Furthermore, differences in datasets, preprocessing approaches and evaluation methods make direct comparison between studies difficult. These gaps provide the motivation for the present study, which evaluates Random Forest, Gradient Boosting and Support Vector Machine using a common preprocessing pipeline, the UNSW-NB15 dataset and a controlled two-virtual-machine environment while considering both classification performance and response latency.

Chapter 3: Methodology

This chapter presents the methods used to design, implement, and evaluate the proposed intrusion detection system. It includes a discussion of the research design,

project scope, data set used, test environment and evaluation criteria, and ethical issues considered.

3.1 Research Design and Approach

The quantitative experimental research design has been adopted because the study evaluates quantitative performance measures, including accuracy, precision, recall and F1-score and detector-side response latency. A Design Science Research (DSR) approach has also been adopted as the project involves the design, implementation, and evaluation of a working artefact, which in this study is the intrusion detection system (vom Brocke, Hevner and Maedche, 2020).

The proposed intrusion detection system was implemented in Python and trained using the UNSW-NB15 dataset. The system was built to classify a specific kind of network traffic as either normal or malicious. To evaluate the system's effectiveness, the evaluation considered both classification performance and response time. As response latency is studied, this enables the suitability of the proposed approach for use in near real-time intrusion detection.

The implementation followed four stages: the preparation of the UNSW-NB15 dataset, preprocessing, model training and system evaluation. Afterward, machine learning models were trained and evaluated and finally integrated into the final prototype. The resulting system was evaluated in a controlled setup. Overall classification performance and response latency were analysed.

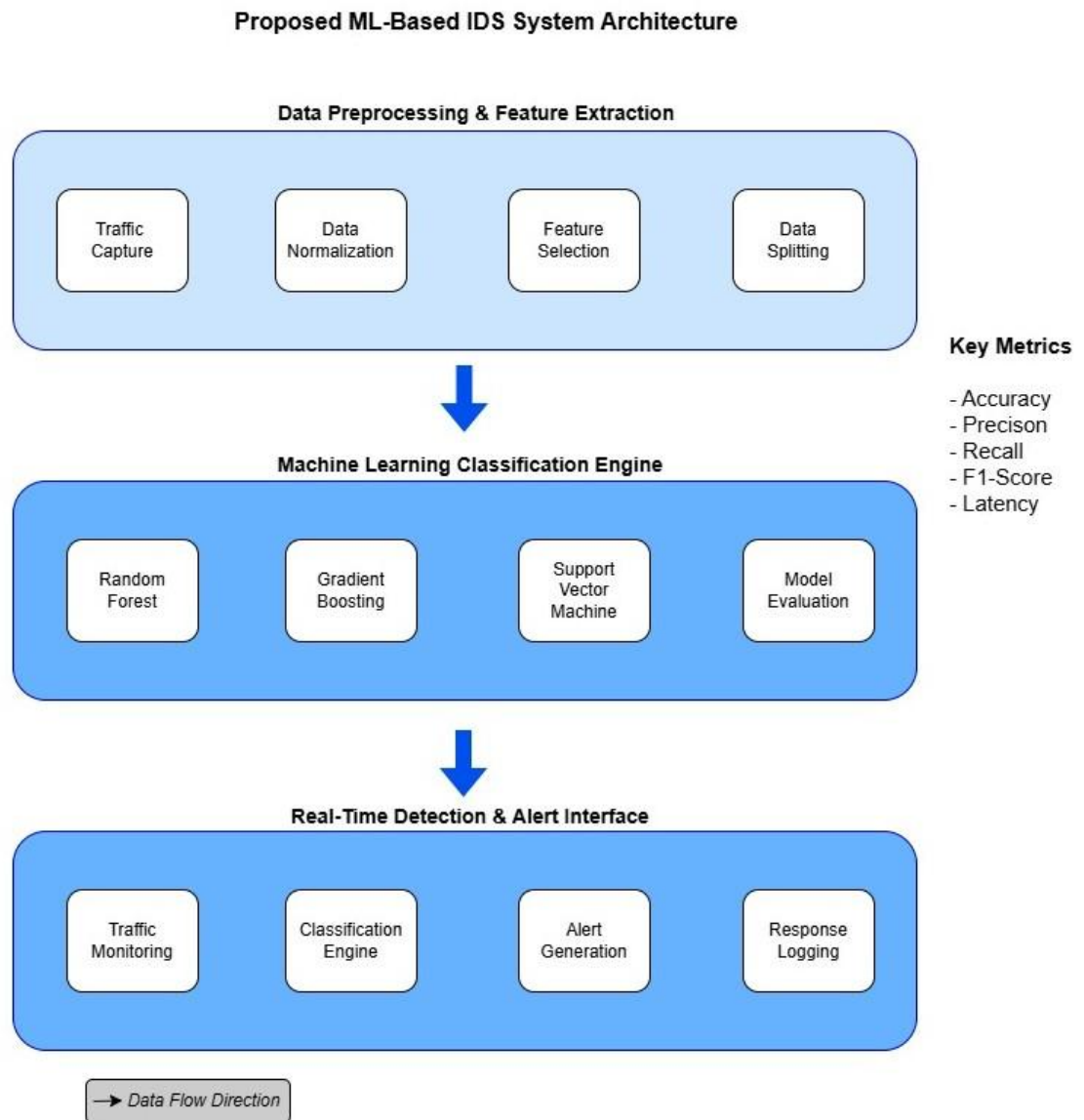


Figure 3.1: Proposed ML-Based IDS System Architecture

Figure 3.1 illustrates the proposed architecture for the intrusion detection system. It consists of the various steps: dataset preparation, model training, traffic classification and performance evaluation. By separating the training phase of the model from the detection phase, the proposed architecture enables evaluation of the detection performance as well as the response time, which are key evaluation criteria.

3.2 Investigation Scope

The investigation scope was defined to maintain project feasibility and support a realistic evaluation of a machine learning-based intrusion detection.

3.2.1 Supervised Machine Learning only

The study was limited to supervised machine learning techniques. The labelled UNSW-NB15 dataset contains normal and malicious network traffic records, making it suitable for supervised classification model training and evaluation (Moustafa and Slay, 2015).

Unsupervised, semi-supervised and deep learning approaches were not considered in the study. Although these approaches can be used for intrusion detection, they would have required additional model development, training and evaluation beyond the scope of this study (Halbouni et al., 2022; Ahmad and AlQahtani, 2023).

3.2.2 Algorithm Selection and Rationale

Three commonly used supervised learning algorithms were selected for comparison: Random Forest (RF), Gradient Boosting (GB) and Support Vector Machine (SVM). These algorithms have been widely applied in intrusion detection research (Alhakeem and Bin Ajlan, 2024; Ahmad and AlQahtani, 2023).

Random Forest was selected because it handles high-dimensional data and provides information on the importance of each feature. Gradient Boosting was included because of its strong predictive performance in classification tasks. SVM was included because it has been widely used in intrusion detection and can separate normal and malicious traffic in complex feature spaces.

Other supervised algorithms, including Naïve Bayes, k-Nearest Neighbours and individual Decision Tree classifiers, were excluded to keep the comparative scope manageable within the available project timeframe.

3.2.3 Dataset scope

The UNSW-NB15 dataset was used to train and evaluate the chosen models. It contains a large number of labelled network traffic records and includes a variety of flow-based features as well as several contemporary attack types (Moustafa and Slay, 2015).

Other datasets, such as CICIDS2017 and CSE-CIC-IDS2018, were not used. Using multiple datasets could provide stronger evidence of model generalisability. However, the different datasets have variations in the number and types of features,

characteristics of the features, preprocessing techniques and label types, which would all require cross-dataset feature alignment.

3.2.4 Scope of the project

The project aimed to implement and evaluate a machine learning-based intrusion detection system within a virtual environment. Binary classification was used to classify the traffic as normal or malicious. The scope of the work undertaken involved data preprocessing, feature engineering, feature selection, model training, traffic classification and performance evaluation.

Intrusion detection and traffic generation were implemented within a two-virtual-machine environment. One virtual machine was configured to simulate traffic, while the second hosted the intrusion detection system. This allows for the detection and response time to be tested in a controlled environment.

Multi-class attack classification, production-network deployment, and automation of response and mitigation, large-scale scalability and Software-Defined Networking integration were excluded from the scope of this work. These areas were excluded to maintain a feasible scope for this project and support detailed comparison of binary classification performance and response latency.

3.2.5 Performance Evaluation Scope

The models were evaluated in terms of accuracy, precision, recall, F1-score and confusion matrices. Training time and prediction time were also measured to compare their computational cost. Detector-side response latency was evaluated within the two-virtual-machine environment. The measurement began after each flow record arrived at VM2 and therefore excluded the network transmission time from VM1 to VM2. However, large-scale scalability, adversarial robustness, long-term operational stability and cost-benefit analysis were excluded in order to maintain a realistic scope for this project.

3.2.6 Summary of Research Scope

This study compared Random Forest, Gradient Boosting and Support Vector Machine for binary network intrusion detection using the UNSW-NB15 dataset. The models

were evaluated using classification-performance metrics and computational-efficiency measurements.

A controlled two-virtual-machine environment was used to assess response latency and the suitability of the selected approach for near real-time detection. The results therefore provide evidence regarding the performance of these algorithms under the specific experimental conditions used in this MSc project.

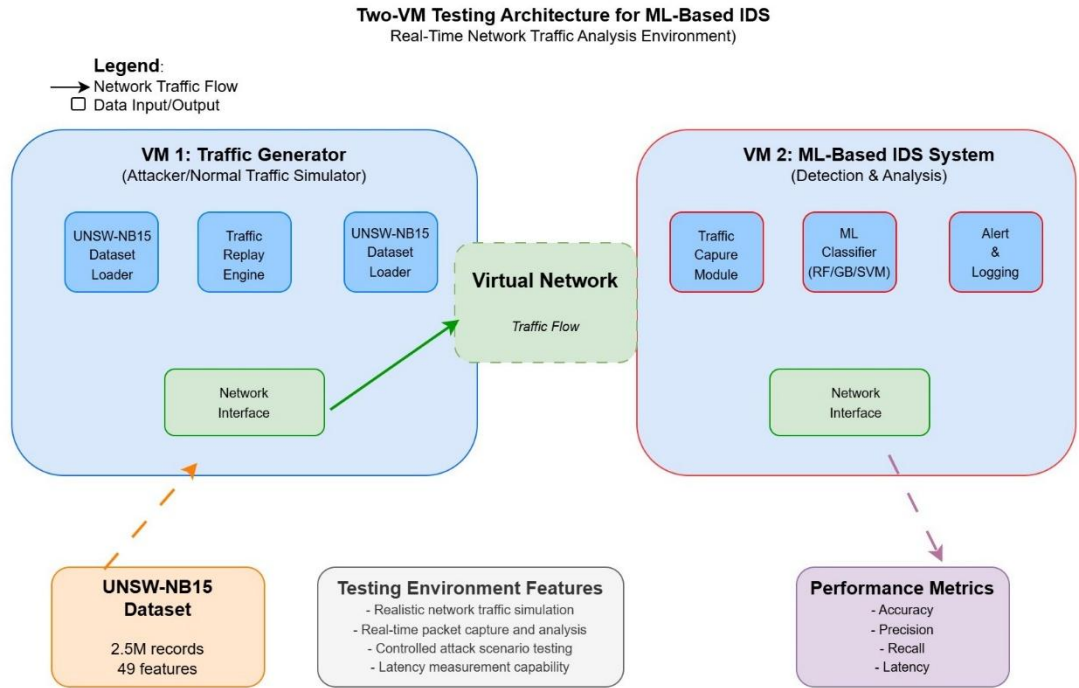


Figure 3.2: Two-VM Testing Environment

The two-VM environment can serve as an adequate testing environment for the proposed ML-based IDS, but it cannot capture all facets of an enterprise network. It is, however, suitable to assess detection accuracy and latency under controlled and repeatable conditions.

3.3 Data Collection and Analysis

The publicly available UNSW-NB15 dataset was used to train and evaluate the machine learning-based intrusion detection system. The dataset contains network traffic records labelled as either normal or malicious. The predefined training and

testing sets were used to support consistency with previous studies and improve reproducibility.

Data were first inspected to determine the structure, feature types, class distribution and data-quality issues. The record identifier was removed because it provided no predictive value, while the attack-category column was excluded to prevent data leakage because binary classification was performed using the label field, where 0 = normal traffic and 1 = malicious traffic.

Categorical features, including protocol, service and connection state, were encoded numerically. Numerical features were normalised to prevent variables with larger ranges from disproportionately influencing model training, particularly the Support Vector Machine algorithm. The dataset was then separated into input features and target labels.

Feature selection was performed using Random Forest feature-importance scores.

Model performance was evaluated using accuracy, precision, recall, F1-score and confusion matrices. Training time, prediction time and response latency were also measured to compare computational efficiency and assess near real-time suitability within the two-virtual-machine environment.

The selected trained model was tested using the predefined UNSW-NB15 testing dataset and integrated into the intrusion detection system prototype.

3.4 Evaluation

The proposed intrusion detection system was evaluated using three criteria: classification performance, computational efficiency and near real-time response latency.

Classification performance was assessed using accuracy, precision, recall, F1-score and confusion matrices. Accuracy indicates the overall proportion of correctly classified network records, precision indicates the proportion of records classified as malicious that were genuinely malicious, recall indicates the proportion of malicious records correctly detected by the model and the F1-score is a balanced measure of precision and recall. Confusion matrices were analysed to understand true positives, true negatives, false positives and false negatives in more detail.

Computational efficiency was also assessed by comparing the training time and prediction time for Random Forest, Gradient Boosting and Support Vector Machine models. This enabled the trade-off between classification performance and computational cost to be assessed.

Near real-time performance was assessed using a two-virtual-machine environment. Traffic records were transmitted from one virtual machine, VM1, to a second virtual machine, VM2, which ran the intrusion detection system. The detector-side latency measurement began after each record arrived at VM2 and ended when the classification result was produced. It therefore included parsing, feature preparation and model prediction, but excluded the network transmission time from VM1 to VM2.

The models were then evaluated and compared using the same predefined UNSW-NB15 testing dataset. The most suitable model was identified based on detection accuracy and response latency under the experimental conditions.

3.5 Ethical Issues

The dataset used in this research contains no personal or organisationally identifiable information and no human participants were involved. All experiments were conducted using the publicly available UNSW-NB15 dataset which consists of labelled normal and attack network records (Moustafa and Slay, 2015).

The evaluation was carried out using a two-virtual-machine system in a controlled and isolated environment. Therefore, no third-party or external organisational systems were targeted during development and evaluation of the intrusion detection system.

The project was conducted solely for defensive cybersecurity purposes. The proposed intrusion detection system was developed to detect malicious network activity rather than support unauthorised access, interference, or other malicious activities. The work was carried out with reference to the UK Computer Misuse Act 1990, which provides for offences relating to unauthorised access to computer systems and unauthorised acts which make it difficult to use computer systems (Great Britain, 1990).

Academic integrity was maintained through appropriate citation of all software tools, dataset, academic publications and frameworks. Experimental procedures and findings, including model limitations, trade-offs and misclassifications, are reported accurately without falsification or omission.

Machine learning-based intrusion detection systems remain susceptible to false-positives and false-negatives. Consequently, the proposed system should be regarded as a decision-support tool rather than a replacement for human security analysts, particularly in environments where accurate incident response is critical.

Chapter 4: Research Design and Implementation

This chapter presents the design and implementation of the proposed intrusion detection system, covering the development environment, data preprocessing, feature selection, model training and controlled two-virtual- machine evaluation.

4.1 Implementation overview

The implementation was divided into four stages: development environment setup, dataset preprocessing, machine learning model training and system testing allowing each component to be developed and tested independently before integration. All work was conducted within controlled virtual-machine environments, with software versions, configurations and experimental outputs documented to support verification and reproducibility.

4.1.1 Technology Stack

Component	Technology Used
Virtualization	Oracle VirtualBox 7.2.8
Operating System	Ubuntu 26.04 LTS
Programming Language	Python 3.14.4
Data Processing	Pandas, NumPy
Machine Learning	Scikit-learn
Visualisation	Matplotlib, Seaborn

A consistent directory structure (data/, models/, results/, scripts/) was used across both virtual machines to organise datasets, scripts, trained models, and evaluation outputs, supporting integration between the two machines.

4.2 Development Environment Setup

4.2.1 Virtual Machine Architecture

Two virtual machines were created for the evaluation. VM1 functioned as the traffic generator and transmitted UNSW-NB15 flow records, while VM2 hosted the intrusion detection system and classified the received records using the trained machine learning model. VM1 was allocated 4GB RAM, 2 CPU cores, and 20GB storage (192.168.56.101); VM2 was allocated 6GB RAM, 2 CPU cores, and 30GB storage (192.168.56.102). Both virtual machines used Ubuntu 26.04 LTS and Python 3.14.4. VM2 received the allocation of memory and storage because it performed the main detection tasks. The machine loaded the trained models, processed the records sent from VM1, and retained the classification outputs and latency measurements.

4.2.2 Network Configuration

Two network adapters were assigned to each virtual machine. Communication between VM1 and VM2 took place through a host-only adapter, while a separate NAT adapter was retained for internet access during software installation. The private connection operated on the 192.168.56.0/24 subnet. Before the experiment runs began, connectivity was checked by sending ICMP ping requests between the two machines. Traffic records were transferred only through the host-only connection. This kept the experimental communication separate from external network traffic and allowed the same arrangement to be maintained across repeated runs. Internet access remained available through the NAT adapter whenever additional packages or updates were required. The complete architecture is presented in Figure 4.1.

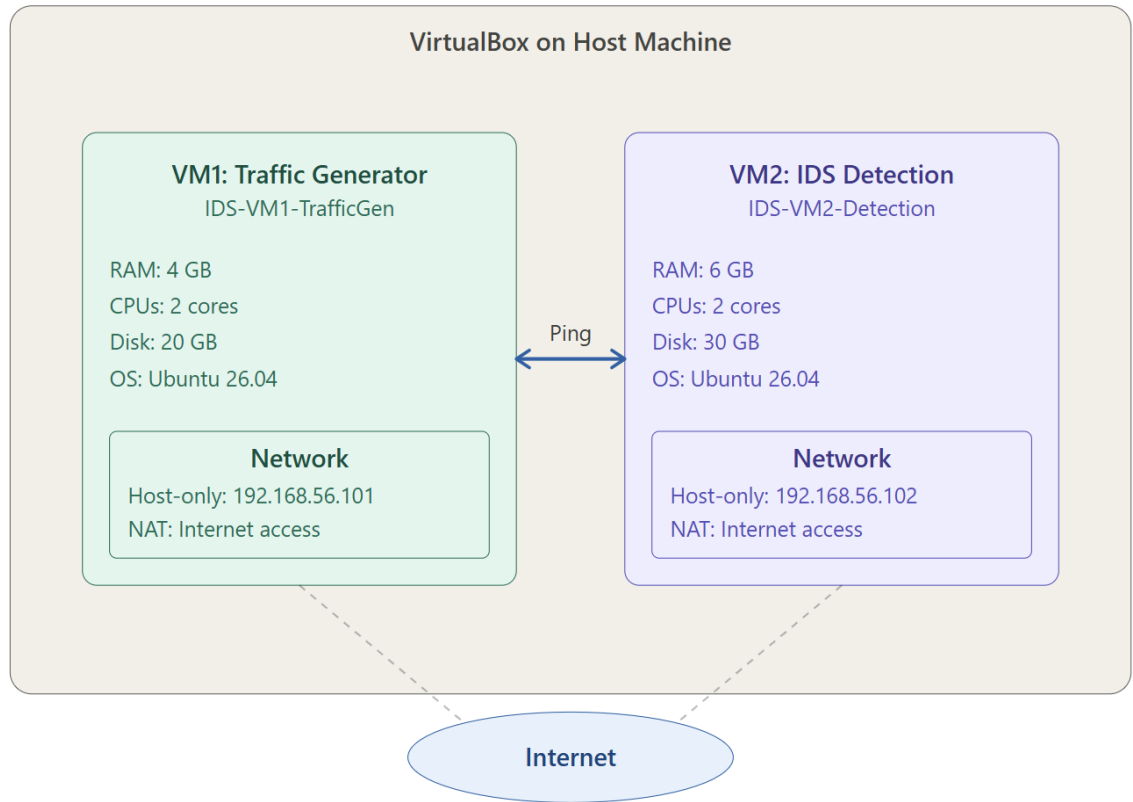


Figure 4.1: Two-virtual-machine Network Architecture

4.2.3 Software Installation and Configuration

Ubuntu 26.04 LTS was installed on both virtual machines through VirtualBox unattended installation option. After installation, both systems were updated before the project dependencies were added. NumPy, Pandas, scikit-learn, Matplotlib and Seaborn were installed from the Ubuntu software repository. This approach was used to maintain compatibility with Python 3.14.4 because suitable third-party wheels were not yet available for some packages. Scapy and Tqdm were installed separately using pip.

The completed environment contained NumPy 2.3.5, Pandas 2.3.3, scikit-learn 1.7.2, Matplotlib 3.10.7, Seaborn 0.13.2, Scapy 2.7.01, Tqdm 4.67.3. These packages supported the main stages of the work, including preprocessing of the dataset, development of the models, visualisation of results, handling of transmitted records and evaluation of the experimental system.

4.2.4 Assessment of Environment Setup

The virtual-machine arrangement made it possible to carry out repeated tests under the same general conditions. However, both machines operated on one physical host and therefore competed for the same processor, memory and storage resources. Consequently, the latency values may differ from those that would be produced by two separate physical systems.

The host-only connection also simplified the network compared with a production environment. Operational traffic may pass through several devices and may be affected by variable delay, packet loss or jitter. These conditions were not reproduced in the present setup. Even with these limitations, the environment remained appropriate for comparing classification performance and detector-side latency under controlled conditions.

4.3 Implementation of the Preprocessing Step

This section explains how the UNSW-NB15 dataset was explored and prepared for model training.

4.3.1 Data Loading and Initial Exploration

The UNSW-NB15 dataset was obtained from the University of New South Wales research repository (Moustafa and Slay, 2015). The predefined training and testing files, `UNSW_NB15_training-set.csv` and `UNSW_NB15_testing-set.csv`, were used because they allow easier comparison with previous studies that used the same dataset and require less processing than the full raw dataset. The files were stored in `~/ml-ids-project/data/`, and a Pandas-based script loaded them (Listing 4.1)

The initial loading code is presented below:

```
import pandas as pd

import numpy as np

DATA_PATH = '/home/mathias/ml-ids-project/data/'

TRAINING_FILE = DATA_PATH + 'UNSW_NB15_training-set.csv'

TESTING_FILE = DATA_PATH + 'UNSW_NB15_testing-set.csv'

train_df = pd.read_csv(TRAINING_FILE)
```

```
test_df = pd.read_csv(TESTING_FILE)
```

Listing 4.1: Initial Data Loading Code

The predefined training dataset contains 82,332 records and 45 columns, while the testing dataset follows the same structure. Although the UNSW-NB15 documentation describes 49 features, the predefined partitions contain 45 columns because dataset creators removed host identifiers such as source and destination IP addresses. This helps reduce the risk of the models learning host-specific patterns instead of general attack behaviour (Moustafa and Slay, 2015). Of the 45 columns, 43 represent flow-based network features, while the remaining two are the labels: `attack_cat`, which identifies the attack type, and `label`, which provides a binary classification (0 = normal, 1 = attack). In line with the scope defined in Section 3.2, this study uses only the binary label field. The dataset consists of 30 integer columns, 11 floating-point columns, and four categorical columns (`proto`, `service`, `state`, `attack_cat`). As most of the features were numerical, only the categorical variables required encoding.

The training set contained 37,000 normal records representing 44.9% of the data, and 45,332 attack records, representing 55.1%. Neither class dominated the dataset to a large extent, so accuracy, precision, recall and F1-score could be interpreted without an extreme class imbalance affecting the results. Nevertheless, the distribution should not be treated as representative of every operational network because legitimate traffic may form a much larger proportion of activity in practice.

The separate attack categories were much less evenly distributed. Generic attacks accounted for 22.9% of the records and Exploits accounted for 13.5%, whereas Shellcode and Worms each represented only 0.5%. A multiclass model would therefore have had far fewer examples from which to learn the least common attacks. Since the investigation focused on binary detection, every attack category was combined into one malicious class. This produced a common basis for evaluating the three algorithms, although it also meant that the resulting system could not identify the specific form of an attack.

Compared with older datasets such as KDD Cup 99, UNSW-NB15 includes attack categories such as Exploits, Reconnaissance and Fuzzers. Its contents therefore reflect a wider range of more recent network threats (Moustafa and Slay, 2015).

The initial quality check found no missing values among the 82,332 records and 45 columns. No records needed to be removed or completed through imputation. Preprocessing could therefore concentrate on encoding the categorical variables, scaling the required features and selecting the inputs used for model training. The outputs from this stage were saved on the project results directory, as shown in Figure 4.2.

```

unsw-nb15@unsw-traffic.com:~$ cat ~/ml-ids-project/results/exploration_output.txt
=====
UNSW-NB15 DATASET EXPLORATION
=====

[1] Loading training data...
    Training set loaded: 82332 rows, 45 columns

[2] Loading testing data...
    Testing set loaded: 175341 rows, 45 columns

[3] Features in dataset:
['id', 'dur', 'proto', 'service', 'state', 'spkts', 'dpkts', 'sbytes', 'dbytes', 'rate', 'sttl', 'dttl', 'sload', 'dload', 'sloss', 'dloss', 'sinpkt', 'dinpkt', 'sjit', 'djitt', 'swin', 'stcpb', 'dtcpb', 'dwin', 'tcprtt', 'synack', 'ackdat', 'sseq', 'dseq', 'trans_depth', 'response_body_len', 'ct_srv_src', 'ct_state_ttl', 'ct_dst_tln', 'ct_src_dport_tln', 'ct_dst_sport_tln', 'ct_dst_src_tln', 'is_ftp_login', 'ct_ftp_cmd', 'ct_flow_http_mthd', 'ct_src_tln', 'ct_srv_dst', 'is_sm_ips_ports', 'attack_cat', 'label']

[4] First 5 rows of training data:
   id  dur  proto  service  ...  ct_srv_dst  is_sm_ips_ports  attack_cat  label
0  1  0.000011  udp  ...  2  0  Normal  0
1  2  0.000008  udp  ...  2  0  Normal  0
2  3  0.000005  udp  ...  3  0  Normal  0
3  4  0.000000  udp  ...  3  0  Normal  0
4  5  0.000010  udp  ...  3  0  Normal  0

[5 rows x 45 columns]

[5] Label distribution in TRAINING set:
    Normal (0): 37000
    Attack (1): 45332

[6] Attack categories distribution:
attack_cat
Normal      37000
Generic     10871
Exploits    11132
Fuzzers      6062
DoS          4089
Reconnaissance 3496
Analysis     677
Backdoor     583
Shellcode    378
Worms        44
Name: count, dtype: int64

[7] Missing values check:
    Total missing values: 0

[8] Data types summary:
int64      30
float64     11
object       4
Name: count, dtype: int64
=====
EXPLORATION / C990 FTP

```

Figure 4.2: UNSW-NB15 Dataset Exploration Output

4.4 Testing Environment Architecture

Near real-time testing was carried out across two virtual machines. VM1 transmitted the flow record, while VM2 received each record and passed it to the intrusion detection model for classification as normal or malicious. VM2 also stored the resulting predictions and the detector-side latency recorded for each processed input. Using the host-only connection kept the transfer path consistent through the evaluation. This made it possible to examine classification accuracy, throughput and response time without changes in the network arrangement between runs. This setup did not reproduce the full complexity of a production network but provided a practical basis for assessing whether the prototype could process records individually within a near real-time context (Imanbayev et al., 2022).

4.5 Data Preprocessing

Preprocessing quality directly affects the performance of the models, training efficiency and generalisability (Alhakeem and Bin Ajlan, 2024). The requirements were determined by the dataset structure and the input requirements of the three algorithms: categorical encoding, feature scaling and removal of non-informative or potentially misleading variables.

4.5.1 Preprocessing Pipeline Implementation

A systematic preprocessing pipeline was implemented in **02_preprocess_data.py**, using Pandas and scikit-learn, applied consistently to both partitions.

Column removal: The id column was removed because it uniquely identifies records rather than describing network behaviour, and attack_cat was removed to avoid target-related information leakage under the binary scope. This left 43 columns (42 predictive features plus the binary label).

Categorical encoding: The features proto, service, and state were converted to numerical form using scikit-learn's LabelEncoder. Training and testing values were combined before encoding to maintain consistent category mappings and avoid errors from categories appearing only in the test set, producing 133 protocol, 13 service, and 11 connection-state categories. Integer encoding was chosen over one-hot encoding to avoid a large increase in dimensionality, separate binary columns for every category would raise memory and computational cost, affecting training and prediction. One limitation of LabelEncoder is that it assigns an arbitrary numerical order to categorical values. This may affect distance-based algorithms such as SVM and was considered when interpreting the comparative results.

Feature scaling: The numerical features were standardised using StandardScaler ($x_scaled = (x - \mu) / \sigma$). The scaler was fitted to the training data using `fit_transform()` and then applied to the test data with `transform()` so that information from the test set did not influence the training process. Standardisation was mainly performed because SVM is sensitive to differences in feature scale. Although Random Forest and Gradient Boosting are generally less affected by feature magnitude, the same standardised dataset was used for all three models to ensure a fair comparison (Imanbayev et al.,

2022). After scaling, the features had approximately zero mean and unit standard deviation.

Persistence: The transformed features and labels were combined and saved as `train_preprocessed.csv` and `test_preprocessed.csv`. Saving the preprocessed datasets separately ensured that each model used the same input data during training and evaluation, helping to keep the experiments consistent and easy to reproduce.

4.5.2 Preprocessing Results

The pipeline converted the raw dataset into a format suitable for machine learning while keeping the original training and testing partitions unchanged (Figure 4.3, Table 4.1). The class distributions were also maintained, with 37,000 normal records (44.9%) and 45,332 attack records (55.1%) in training set, and 56,000 normal (31.9%) and 119,341 attack records (68.1%) in the test set. The larger proportion of attack traffic in the test set creates a more challenging evaluation. Because the test data are imbalanced, model performance is assessed using class-specific metrics, including precision, recall, F1-score and the confusion matrix, rather than relying on accuracy alone.

```

mathias@IDS-VM1-TrafficGen:~$ cat ~/ml-ids-project/results/preprocessing_output.txt
=====
UNSW-NB15 DATA PREPROCESSING
=====

[1] Loading training and testing data...
    Training: 82332 rows, 45 columns
    Testing: 175341 rows, 45 columns

[2] Categorical columns (need encoding):
    proto: 131 unique values
        Examples: ['udp', 'arp', 'tcp', 'igmp', 'ospf']
    service: 13 unique values
        Examples: ['- ', 'http', 'ftp', 'ftp-data', 'smtp']
    state: 7 unique values
        Examples: ['INT', 'FIN', 'REQ', 'ACC', 'CON']

[3] Dropping unnecessary columns...
    Dropped: ['id', 'attack_cat']
    Remaining columns: 43

[4] Encoding categorical features to numbers...
    proto encoded: 133 categories -> numbers
    service encoded: 13 categories -> numbers
    state encoded: 11 categories -> numbers

[5] Separating features (X) and labels (y)...
    X_train: (82332, 42)
    y_train: (82332,)

[6] Normalizing features (StandardScaler)...
    Features normalized to mean=0, std=1

[7] Saving preprocessed data...
    Training data saved: /home/mathias/ml-ids-project/data/train_preprocessed.csv
    Testing data saved: /home/mathias/ml-ids-project/data/test_preprocessed.csv

[8] Preprocessing Summary:
    Original training columns: 45
    Preprocessed training columns: 43
    Training samples: 82332
    Testing samples: 175341
    All features are now numeric and normalized

[9] Class Distribution:
    Training - Normal (0): 37000
    Training - Attack (1): 45332
    Testing - Normal (0): 56000
    Testing - Attack (1): 119341

=====
PREPROCESSING COMPLETE!
=====
mathias@IDS-VM1-TrafficGen:~$

```

Figure 4.3: Preprocessing Pipeline Output Results

Table 4.1 summarises the transformations applied:

Preprocessing Aspect	Value
Original training columns	45
Original testing columns	45
Columns removed	2 (id, attack_cat)
Final columns after preprocessing	43
Categorical features encoded	3 (proto, service, state)
Total unique protocols	133

Total unique services	13
Total unique states	11
Normalisation method	StandardScaler (mean=0, std=1)
Training samples	82,332
Testing samples	175,341
Training class balance	44.9% Normal / 55.1% Attack
Testing class balance	31.9% Normal / 68.1% Attack

Table 4.1: Preprocessing Pipeline Summary

4.5.3 Critical Reflection on the Preprocessing

Two preprocessing decisions should be considered. First, LabelEncoder was used instead of one-hot encoding to reduce the number of generated features and limit the computational cost of near real-time detection. One-hot encoding may produce better classification results and could be investigated in future. Second, the encoder was fitted using categories from both the training and testing partitions to ensure that the same numerical values were assigned across both datasets. This introduced limited information sharing between the partitions, although no class-label information was used. It is therefore unlikely to have affected the reported evaluation results. However, a production system would still need a clear method for handling categories that were not present during training.

No further data cleaning or class balancing was carried out because the dataset was already labelled and contained no missing values. In a real network, incomplete flow records, packet loss and sensor errors may produce missing data, so suitable imputation methods would need to be considered. Future experiments could also compare StandardScaler with alternatives such as RobustScaler and MinMaxScaler.

The preprocessing outputs were checked before model training. The saved files retained the expected numbers of rows and columns, as well as the original class distributions. All model input features were numerical, the scaled features had approximately zero mean and unit standard deviation, and the encoded categorical variables contained integer values within the expected ranges.

4.6 Feature Selection

Feature selection was performed after preprocessing to determine which traffic variables contributed most strongly to intrusion detection. Retaining a smaller group of

informative variables can reduce unnecessary computation, shorten training and prediction time, and make the resulting model easier to interpret (Alhakeem and Bin Ajlan, 2024).

4.6.1 Rationale and Methodology

Following preprocessing, 42 predictive features remained in the dataset. Their contributions were unlikely to be equal, and retaining all of them could have increased memory use and prediction time without providing a corresponding improvement in detection. A reduced set was therefore preferred because the study aimed to support near real-time processing. Removing features with limited or repetitive information could also reduce redundancy and lower the risk of overfitting.

Random Forest feature importance was used because it is calculated as part of the model training process and did not require a separate feature-selection method. During training, the model measures how much each feature reduces prediction impurity when it is used to split the data, and the importance scores are then averaged across all trees (Pedregosa et al., 2011). The method can capture non-linear relationships, is not affected by differences in feature scale, and produces scores that can be compared across features.

Other methods were considered but were less suitable for this study. The Chi-square test requires non-negative values, which was not compatible with the output from StandardScaler. Recursive Feature Elimination would have required considerably more computation across 42 features, L1 regularisation is based on a linear relationship between the features and the target. PCA was also not selected because it combines the original features into new components, making the results more difficult to interpret. Random Forest importance therefore provided a practical balance between computational cost and interpretability.

4.6.2 Implementation

Feature selection was carried out in `03_feature_selection.py` using `RandomForestClassifier` with 100 trees, a maximum depth of 15 and `n_jobs=-1`. The feature-importance scores were extracted and ranked from highest to lowest. The top 20 features, representing approximately 48% of the original feature set, were retained, while remaining 22 lower-ranked features were

removed (Figure 4.4). Table 4.2 presents the ten highest-ranked features, while Figure 4.5 shows the importance scores for all 20 selected features.

```

mathias@IDS-VM1-TrafficGen:~$ python3 ~/ml-ids-project/scripts/03_feature_selection.py
=====
FEATURE SELECTION - Random Forest Importance
=====

[1] Loading preprocessed data...
    Training: (82332, 43)
    Testing:  (175341, 43)

[2] Separating features and labels...
    Features: 42

[3] Training Random Forest to calculate feature importance...
    (This may take 2-3 minutes...)
    Training complete!

[4] Calculating feature importance...

    Top 10 most important features:
    -----
    ct_dst_src_ltm           0.0946
    ct_state_ttl             0.0827
    sload                    0.0628
    sttl                     0.0586
    sbytes                   0.0583
    rate                     0.0580
    smean                    0.0544
    ct_srv_dst               0.0512
    ct_dst_sport_ltm         0.0508
    dbytes                   0.0395

[5] Selecting top 20 features...
    Selected 20 features

[6] Creating reduced datasets...

[7] Saving reduced datasets...
    Saved: /home/mathias/ml-ids-project/data/train_selected.csv
    Saved: /home/mathias/ml-ids-project/data/test_selected.csv
    Saved: /home/mathias/ml-ids-project/results/feature_importance.csv

[8] Creating feature importance chart...
    Chart saved: /home/mathias/ml-ids-project/results/feature_importance_chart.png

[9] Feature Selection Summary:
    Original features: 42
    Selected features: 20
    Reduction: 22 features removed
    Training samples: 82332
    Testing samples: 175341

=====
FEATURE SELECTION COMPLETE!
=====

```

Figure 4.4: Feature Selection Script Output

Rank	Feature Name	Importance Score	Description
1	ct_dst_src_ltm	0.0946	Count of connections with same destination and source

2	ct_state_ttl	0.0827	Count of state and TTL combinations
3	sload	0.0628	Source-to-destination data rate (bits/second)
4	sttl	0.0586	Source time-to-live value
5	sbytes	0.0583	Total bytes from source to destination
6	rate	0.0580	Transmission rate
7	smean	0.0544	Mean packet size from source
8	ct_srv_dst	0.0512	Count of same service and destination
9	ct_dst_sport_ltm	0.0508	Count of same destination and source port
10	dbytes	0.0395	Total bytes from destination to source

Table 4.2: Top 10 Most Important Features

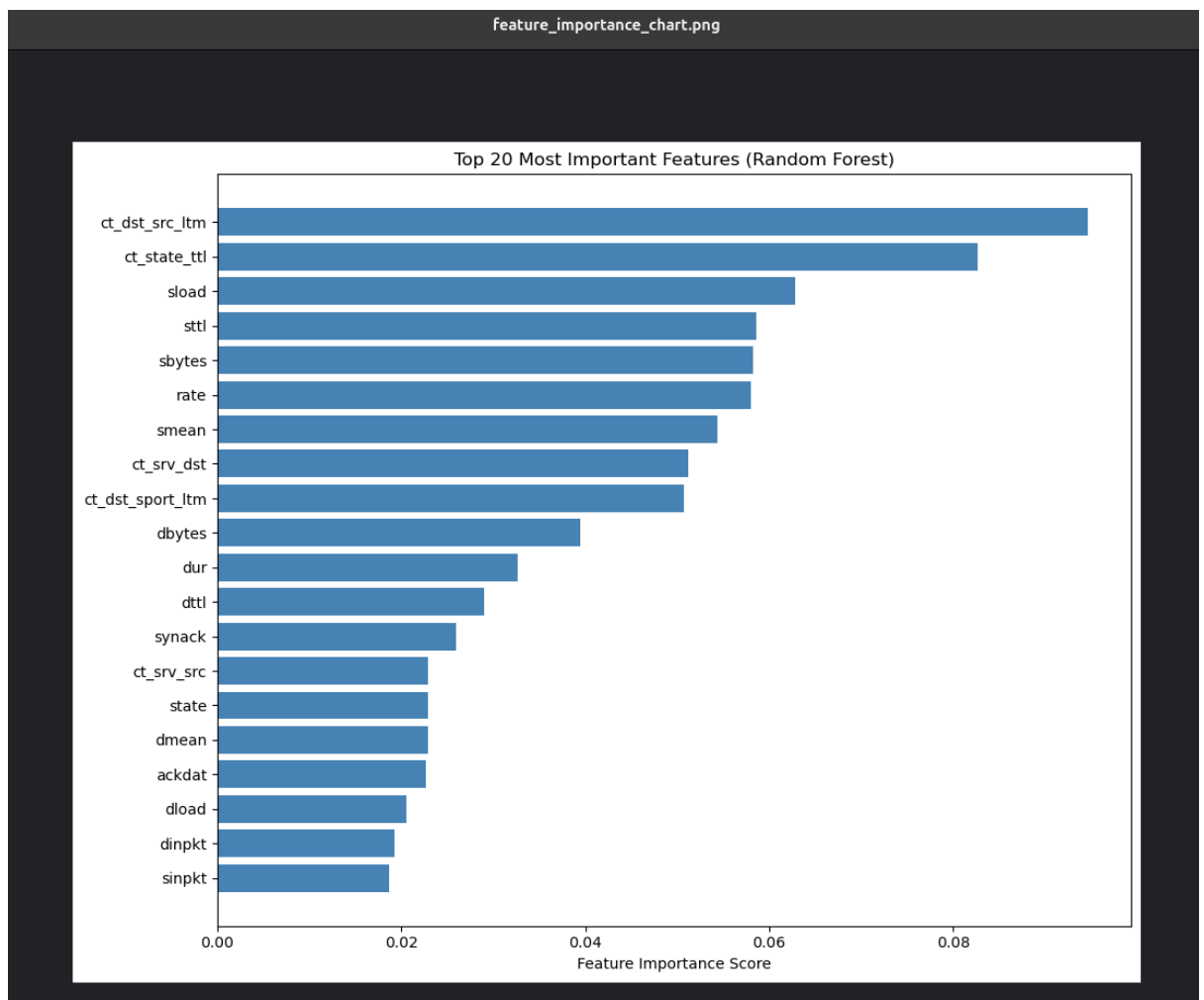


Figure 4.5: Top 20 Feature Importance Scores (Random Forest)

The highest-ranked features were mainly related to connection activity, traffic volume and protocol behaviour. Connection-based features included `ct_dst_src_ltm` and `ct_state_ttl`, while traffic-volume features included `sload`, `sbytes`, `dbytes`, and `smean`. The `sttl` feature also captured protocol-level behaviour. These features are relevant to intrusion detection because denial-of-service and port-scanning attacks can produce unusual connection counts and traffic volumes. Unusual TTL values may also suggest spoofed addresses or uncommon network routes, while service and port-related features such as `ct_srv_dst` and `ct_dst_sport_ltm` can help identify attacks directed at services such as HTTP or SSH. The fact that the selected features reflected known indicators of malicious activity provided further support for the feature-selection approach.

Feature selection reduced the number of predictive features from 42 to 20, representing a 52.4% reduction. This was expected to reduce training time, memory

usage, and prediction time, which supports the near real-time use aim of the study. However, removing lower-ranked features may also result in some useful information being lost. The reduced datasets were saved as `train_selected.csv` and `test_selected.csv` so that the same feature set could be used consistently across all three models.

4.7 Machine Learning Model Implementation

This section describes the implementation and evaluation of the three supervised machine-learning algorithms. Each model was trained on `train_selected.csv` (82,332 records, 20 features) and evaluated on `test_selected.csv` (175,341 records), using accuracy, precision, recall, F1-score, ROC AUC, confusion matrices, and training and prediction time, enabling a consistent comparison of both classification performance and computational efficiency.

4.7.1 Random Forest Implementation

Random Forest was implemented first. It suits intrusion detection because it models non-linear relationships, handles high-dimensional data and provides feature-importance scores, combining multiple decision trees to reduce overfitting relative to a single tree (Ahmad and AlQahtani, 2023).

Configuration: `RandomForestClassifier` was configured with 100 estimators, `max_depth=20`, `min_samples_split=5`, `min_samples_leaf=2`, `max_features='sqrt'`, `random_state=42`, and `n_jobs=-1`. These values balanced classification performance, overfitting control, and training time within the available resources.

Training and prediction: Training completed in 4.29 seconds, showing that the model could be retrained periodically without a high computational cost. Prediction across all 175,341 test records took 0.53 seconds. This is equivalent to approximately 0.0030 ms per record, or about 331,000 records per second when processed in a batch. However, this result reflects batch prediction only and does not include the additional per-record processing carried out in the two-virtual-machine environment (Figure 4.6).

```
mathias@IDS-VMI-TrafficGen:~$ nano ~/ml-ids-project/scripts/04_train_random_forest.py
mathias@IDS-VMI-TrafficGen:~$ python3 ~/ml-ids-project/scripts/04_train_random_forest.py
=====
RANDOM FOREST CLASSIFIER - TRAINING & EVALUATION
=====

[1] Loading feature-selected data...
    Training: (82332, 21)
    Testing: (175341, 21)

[2] Preparing features and labels...
    Features: 20
    Training samples: 82332
    Testing samples: 175341

[3] Configuring Random Forest classifier...
    n_estimators: 100
    max_depth: 20
    min_samples_split: 5
    min_samples_leaf: 2

[4] Training Random Forest model...
    (This may take 3-5 minutes...)
    Training complete in 4.29 seconds

[5] Making predictions on test set...
    Predictions complete in 0.53 seconds
    Average prediction time: 0.0030 ms per sample

[6] Calculating performance metrics...
    Accuracy: 0.8975 (89.75%)
    Precision: 0.9888 (98.88%)
    Recall: 0.8598 (85.98%)
    F1-Score: 0.9194 (91.94%)
    ROC AUC: 0.9858

[7] Confusion Matrix:
    True Negatives (Normal correctly identified): 54844
    False Positives (Normal misclassified as Attack): 1156
    False Negatives (Attack missed): 16825
    True Positives (Attack correctly detected): 102516

[8] Classification Report:
              precision    recall  f1-score   support

   Normal      0.77       0.98       0.86     56000
   Attack      0.99       0.86       0.92    119341

   accuracy          0.89       0.90       0.90    175341
  macro avg       0.88       0.92       0.89    175341
 weighted avg     0.92       0.90       0.90    175341

[9] Creating confusion matrix visualization...
    Confusion matrix saved: /home/mathias/ml-ids-project/results/rf_confusion_matrix.png

[10] Saving trained model...
    Model saved: /home/mathias/ml-ids-project/models/random_forest_model.pkl

[11] Saving results summary...
    Results saved: /home/mathias/ml-ids-project/results/random_forest_results.txt

=====
RANDOM FOREST TRAINING COMPLETE!
=====
```

Figure 4.6: Random Forest Training and Evaluation Terminal Output

Performance Metric	Value
Accuracy	89.75%
Precision	98.88%
Recall	85.90%
F1-Score	91.94%
ROC AUC Score	0.9858
Training Time	4.29 seconds
Prediction Time (Total)	0.53 seconds
Prediction Time (Per Sample)	0.0030 milliseconds

Table 4.3: Random Forest Performance Metrics on UNSW-NB15 Test Set

The model correctly classified approximately 90% of the test records. Its precision of 98.88% shows that when the model identified traffic as malicious, it was almost always correct, resulting in very few false alarms. The recall value of 85.90% means that the model failed to identify approximately 14% of the attack records. The model therefore performed more strongly at avoiding false alarms than at detecting every malicious instance. The F1-score of 91.94% indicates a good balance between precision and recall, while the ROC AUC of 0.9858 shows that the model was highly effective at separating normal and malicious traffic across different classification thresholds.

The confusion matrix (Figure 4.7) shows 54,844 true negatives, 1,156 false positives, 16,825 false negatives, and 102,516 true positives. The false-positive rate of approximately 2.06% confirms effective recognition of legitimate traffic, consistent with the high precision. The 16,825 missed attacks are more significant from a security perspective, confirming that the model's principal weakness was missed attacks rather than excessive false alarms; the pattern reflected in the gap between precision (98.88%) and recall (85.90%).

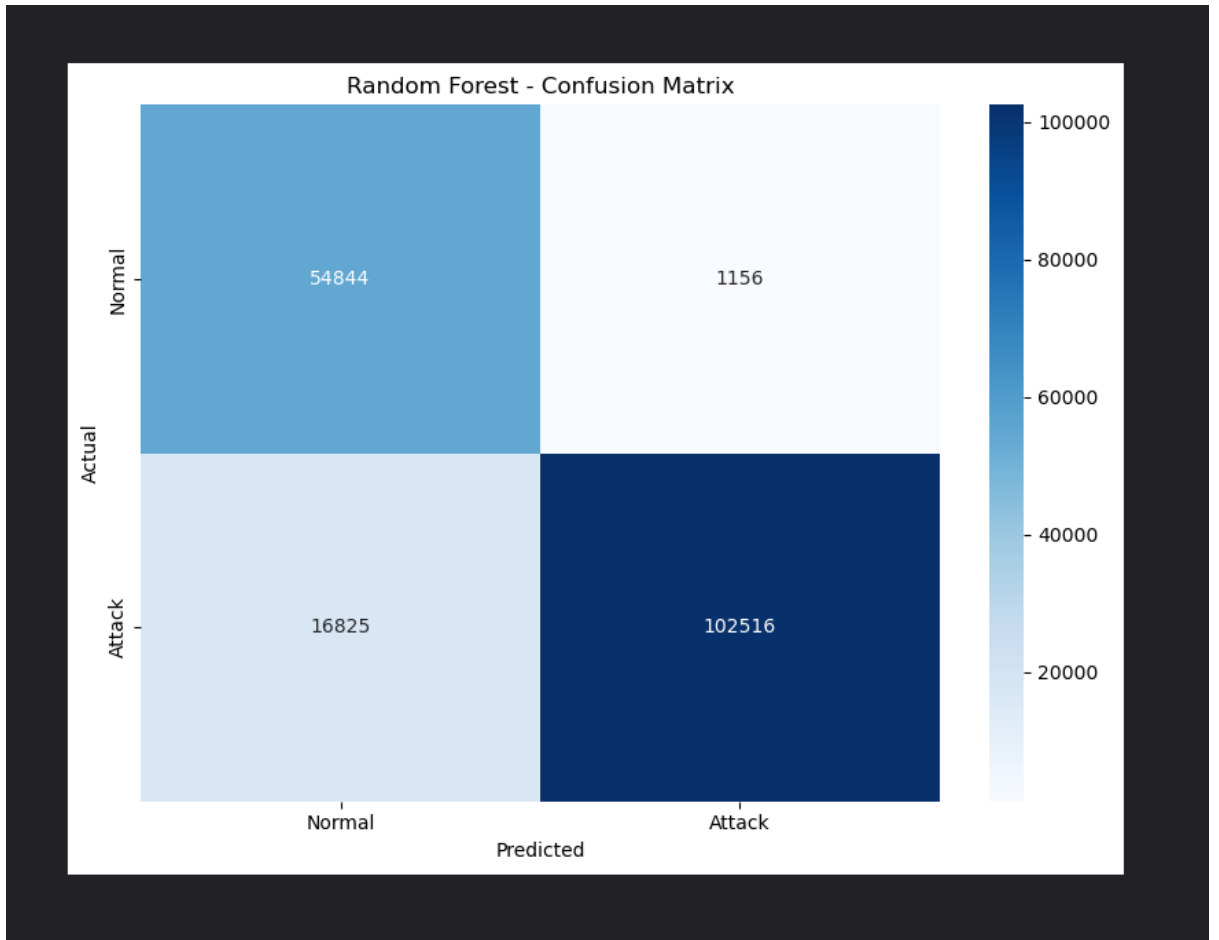


Figure 4.7: Random Forest Confusion Matrix

Overall, Random Forest combined strong detection performance, rapid prediction, and efficient training, making it a strong candidate for the two-VM evaluation. A deployment prioritising attack detection could adjust the classification threshold or apply cost-sensitive learning to improve recall, potentially at the cost of more false positives. The trained model was serialised with joblib as `random_forest_model.pkl`, allowing it to be transferred to VM2 and loaded for real-time classification without retraining.

4.7.2 Gradient Boosting Implementation

Gradient Boosting builds trees sequentially, each trained to reduce the errors of previous stages (Ahmad and AlQahtani, 2023). `GradientBoostingClassifier` was configured as for Random Forest (`min_samples_split=5`, `min_samples_leaf=2`, `random_state=42`) with 100 boosting stages, `learning_rate=0.1`, and `max_depth=5` (shallower trees suited to boosting's weak learners). Unlike Random Forest, it does not use `n_jobs`, because its sequential trees cannot be trained in parallel which affects training time.

Training took 46.25 seconds, roughly eleven times longer than Random Forest, reflecting the sequential construction. Prediction over the test set took 0.30 seconds - 0.0017 ms per record, marginally faster than Random Forest, likely because its shallower trees (depth 5 versus 20) require fewer nodes to traverse, though this was not tested separately (Figure 4.8).

```

mathias@IDS-VM1-TrafficGen:~$ nano ~/ml-ids-project/scripts/05_train_gradient_boosting.py
mathias@IDS-VM1-TrafficGen:~$ python3 ~/ml-ids-project/scripts/05_train_gradient_boosting.py | tee ~/ml-ids-project/results/gb_output.txt
=====
GRADIENT BOOSTING CLASSIFIER - TRAINING & EVALUATION
=====

[1] Loading feature-selected data...
    Training: (82332, 21)
    Testing: (175341, 21)

[2] Preparing features and labels...
    Features: 20
    Training samples: 82332
    Testing samples: 175341

[3] Configuring Gradient Boosting classifier...
    n_estimators: 100
    learning_rate: 0.1
    max_depth: 5
    min_samples_split: 5
    min_samples_leaf: 2

[4] Training Gradient Boosting model...
    (This is slower than Random Forest - may take 5-10 minutes...)
    Training complete in 46.25 seconds

[5] Making predictions on test set...
    Predictions complete in 0.30 seconds
    Average prediction time: 0.0017 ms per sample

[6] Calculating performance metrics...
    Accuracy: 0.8986 (89.86%)
    Precision: 0.9886 (98.86%)
    Recall: 0.8609 (86.09%)
    F1-Score: 0.9204 (92.04%)
    ROC AUC: 0.9864

[7] Confusion Matrix:
    True Negatives (Normal correctly identified): 54820
    False Positives (Normal misclassified as Attack): 1180
    False Negatives (Attack missed): 16601
    True Positives (Attack correctly detected): 102740

[8] Classification Report:
      precision    recall  f1-score   support

   Normal      0.77      0.98      0.86     56000
   Attack      0.99      0.86      0.92    119341

   accuracy          0.90    175341
  macro avg      0.88      0.92      0.89    175341
 weighted avg      0.92      0.90      0.90    175341

[9] Creating confusion matrix visualization...
    Confusion matrix saved: /home/mathias/ml-ids-project/results/gb_confusion_matrix.png

[10] Saving trained model...
    Model saved: /home/mathias/ml-ids-project/models/gradient_boosting_model.pkl

[11] Saving results summary...
    Results saved: /home/mathias/ml-ids-project/results/gradient_boosting_results.txt

=====
GRADIENT BOOSTING TRAINING COMPLETE!
=====
mathias@IDS-VM1-TrafficGen:~$

```

Figure 4.8: Gradient Boosting Training and Evaluation Script Output

The Gradient Boosting classifier's performance on the test set is presented in Table 4.4.

Performance Metric	Value
--------------------	-------

Accuracy	89.86%
Precision	98.86%
Recall	86.09%
F1-Score	92.04%
ROC AUC Score	0.9864
Training Time	46.25 seconds
Prediction Time (Total)	0.30 seconds
Prediction Time (Per Sample)	0.0017 milliseconds

Table 4.4: Gradient Boosting Performance Metrics on UNSW-NB15 Test Set

Gradient Boosting achieved marginally higher accuracy (89.86%), F1-score (92.04%), recall (86.09%), and ROC AUC (0.9864) than Random Forest, with comparable precision (98.86%). The confusion matrix (Figure 4.9) shows 54,820 true negatives, 1,180 false positives, 16,601 false negatives, and 102,740 true positives, 224 more attacks detected than Random Forest at the cost of 24 additional false positives, a small but favourable trade-off, though 16,601 missed attacks remained the main weakness.

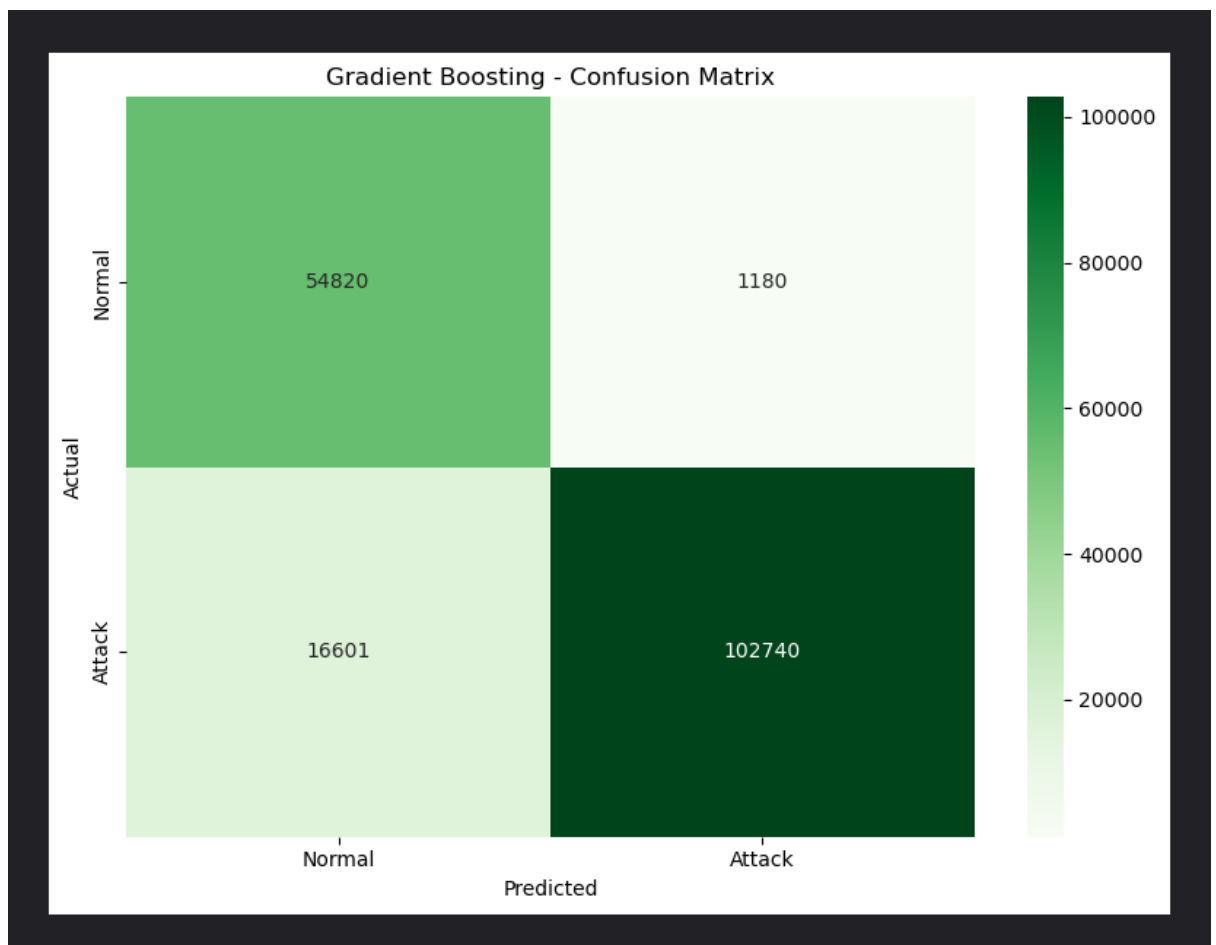


Figure 4.9: Gradient Boosting Confusion Matrix

The improvement in classification performance over Random Forest was small, at approximately 0.1 to 0.2 percentage points, whereas training required around eleven times longer. This suggests a trade-off rather than a clear advantage. Gradient Boosting may be a better choice when achieving the highest possible classification performance is the priority, while Random Forest may be more suitable when faster retraining and simpler deployment are more important. The model was serialised as `gradient_boosting_model.pkl` and retained as an alternative candidate for real-time evaluation.

4.7.3 Support Vector Machine Implementation

SVM identifies a decision boundary separating normal and attack traffic in the feature space (Alhakeem and Bin Ajan, 2024), providing a non-ensemble comparison. SVC was configured with an RBF kernel, $C=1.0$, $\gamma='scale'$, and $probability=True$ (to enable ROC AUC). Feature standardisation from preprocessing was particularly important here, as SVM is sensitive to feature magnitude.

Training took 800.67 seconds (approximately 13.3 minutes) around 187 times longer than Random Forest and 17 times longer than Gradient Boosting reflecting SVM's poor scaling with training-set size, an important limitation for large intrusion detection datasets. Prediction over the test set took 158.11 seconds, or 0.9017 ms per record: roughly 300 times slower than Random Forest and 530 times slower than Gradient Boosting. Although under one millisecond in isolation, this leaves less processing time for receiving traffic and extracting features in an operational system (Figure 4.10).

```

mathias@IDS-VM1-TrafficGen:~$ nano ~/ml-ids-project/scripts/06_train_svm.py
mathias@IDS-VM1-TrafficGen:~$ python3 ~/ml-ids-project/scripts/06_train_svm.py | tee ~/ml-ids-project/results/svm_output.txt
=====
SUPPORT VECTOR MACHINE CLASSIFIER - TRAINING & EVALUATION
=====

[1] Loading feature-selected data...
    Training: (82332, 21)
    Testing:  (175341, 21)

[2] Preparing features and labels...
    Features: 20
    Training samples: 82332
    Testing samples: 175341

[3] Configuring Support Vector Machine classifier...
    kernel: rbf (radial basis function)
    C: 1.0
    gamma: scale
    probability: True

[4] Training SVM model...
    (SVM is the slowest algorithm - this may take 20-40 minutes...)
    (Please be patient and do not close the terminal)
    Training complete in 800.67 seconds (13.3 minutes)

[5] Making predictions on test set...
    Predictions complete in 158.11 seconds
    Average prediction time: 0.9017 ms per sample

[6] Calculating performance metrics...
    Accuracy:  0.8793 (87.93%)
    Precision: 0.9813 (98.13%)
    Recall:    0.8386 (83.86%)
    F1-Score:  0.9044 (90.44%)
    ROC AUC:   0.9593

[7] Confusion Matrix:
    True Negatives (Normal correctly identified): 54094
    False Positives (Normal misclassified as Attack): 1906
    False Negatives (Attack missed): 19262
    True Positives (Attack correctly detected): 100079

[8] Classification Report:
      precision    recall  f1-score   support

   Normal         0.74         0.97         0.84       56000
   Attack         0.98         0.84         0.90      119341

 accuracy          0.88       175341
  macro avg         0.86         0.90         0.87       175341
 weighted avg         0.90         0.88         0.88       175341

[9] Creating confusion matrix visualization...
mathias@IDS-VM1-TrafficGen:~$

```

Figure 4.10: Support Vector Machine Training and Evaluation Script Output

The SVM classifier's performance on the test set is presented in Table 4.5.

Performance Metric	Value
Accuracy	87.93%
Precision	98.13%
Recall	83.86%
F1-Score	90.44%
ROC AUC Score	0.9593
Training Time	800.67 seconds (13.3 minutes)
Prediction Time (Total)	158.11 seconds
Prediction Time (Per Sample)	0.9017 milliseconds

Table 4.5: Support Vector Machine Performance Metrics on UNSW-NB15 Test Set

SVM produced the lowest accuracy (87.93%), F1-score (90.44%), recall (83.86%), and ROC AUC (0.9593) of the three models, while retaining high precision (98.13%). The confusion matrix (Figure 4.11) shows 54,094 true negatives, 1,906 false positives, 19,262 false negatives, and 100,079 true positives, more errors of both types than either ensemble method (19,262 missed attacks versus 16,825 and 16,601; 1,906 false alarms versus 1,156 and 1,180).

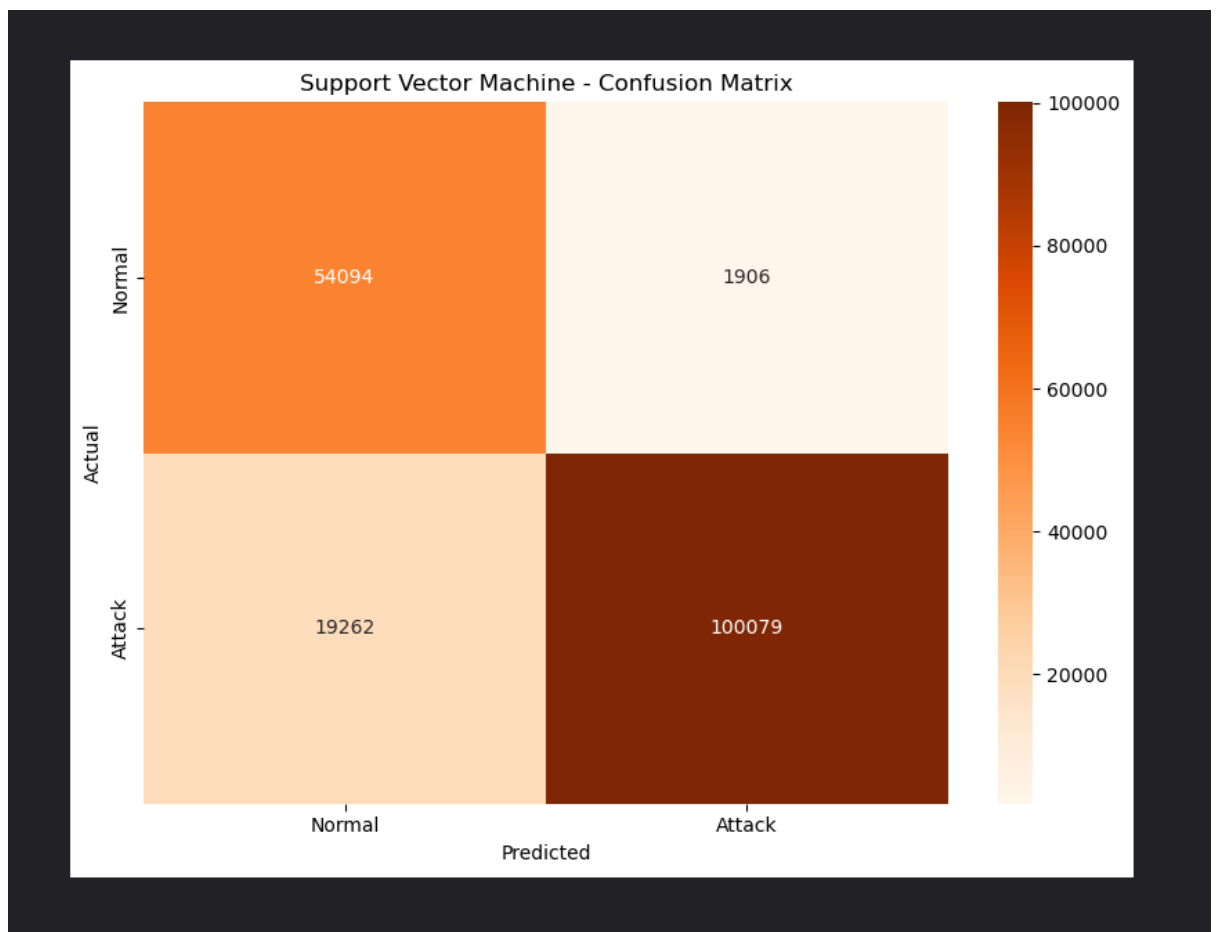


Figure 4.11: Support Vector Machine Confusion Matrix

SVM therefore achieved the weakest classification performance at the greatest computational cost. Default C and gamma values were used rather than extensive optimisation, which might improve results at further computational expense. Despite underperforming, including SVM strengthened the comparison, demonstrating that the strong ensemble results were not matched by every supervised method and providing a clear example of the accuracy–efficiency trade-off. The model was serialised as `svm_model.pkl` but, given its lower accuracy and much slower prediction, was not selected for the real-time evaluation.

4.7.4 Comparative Analysis of Machine Learning Models

Following the separate evaluation of Random Forest, Gradient Boosting and Support Vector Machine, this section compares their classification performance and computational efficiency. Both areas are important because an intrusion detection

model must classify traffic accurately while also producing results quickly enough for near real-time use.

Table 4.6 presents the classification and computational results for the three algorithms side by side, allowing a direct comparison.

Metric	Random Forest	Gradient Boosting	Support Vector Machine
Accuracy	89.75%	89.86%	87.93%
Precision	98.88%	98.86%	98.13%
Recall	85.90%	86.09%	83.86%
F1-Score	91.94%	92.04%	90.44%
ROC AUC	0.9858	0.9864	0.9593
Training Time (seconds)	4.29	46.25	800.67
Prediction (ms per sample)	0.0030	0.0017	0.9017
Attack Missed (FN)	16,825	16,601	19,262

Table 4.6: Comparative Performance of Random Forest, Gradient Boosting, and SVM

In classification, Random Forest and Gradient Boosting were closely matched and both outperformed SVM. All three exceeded 98% precision, so the main differentiator was recall, where the ensemble methods detected more attacks. The differences in computational efficiency were far larger than the differences in accuracy: Random Forest trained in 4.29 seconds versus 46.25 for Gradient Boosting and 800.67 for SVM and predicted at 0.0030 ms/record versus 0.0017 (Gradient Boosting) and 0.9017 (SVM). These differences matter for near real-time detection, where the model must classify continuous traffic without a processing backlog.

Both ensemble methods were therefore more suitable for near real-time detection than SVM, which combined the lowest accuracy with the highest cost. Random Forest was selected for the two-VM evaluation: although Gradient Boosting achieved marginally higher classification results, the difference was small while Random Forest trained far faster and still predicted extremely quickly. Gradient Boosting remained a suitable alternative but was not selected for the final system, while SVM was excluded because of its weaker overall performance. This selection demonstrates that algorithm choice

for intrusion detection cannot rest on accuracy alone, computational efficiency must also be considered, as an accurate model is unsuitable if it cannot process traffic quickly enough.

4.8 Two-Virtual-Machine Real-Time Detection Evaluation

Batch evaluation alone could not show how the system behaves when records are received and classified individually. A two-VM experiment therefore assessed whether Random Forest could classify streamed flow records with low enough latency for near real-time operation. It simulated streamed traffic under controlled conditions and did not perform live packet capture or feature extraction from raw packets.

4.8.1 Architecture and Setup

Using the architecture from Section 4.4, a script on VM1 loaded selected test records and transmitted them individually to VM2 over a TCP socket, simulating a continuous input stream; a script on VM2 received each record, prepared it, and classified it using the saved Random Forest model, recording arrival and classification times. The `random_forest_model.pkl` file was copied to VM2 using Secure Copy over the host-only network.

The experiment used 1,000 randomly selected test records (fixed seed for repeatability). The first 10 were treated as warm-up and excluded to reduce the influence of one-time start-up activities, leaving 990 flows in the latency analysis. Two latencies were recorded: **classification latency** (model prediction start to classification result) and **detector-side processing latency** (record arrival at VM2 to classification result, including parsing and preparation). Figure 4.12 shows the detector's successful execution.


```

mathias@IDS-VM2-Detection:~$ python3 ~/ml-ids-project/scripts/07_ids_detector.py
=====
IDS DETECTOR (VM2) - REAL-TIME CLASSIFICATION
=====

[1] Loading Random Forest model...
    Model loaded successfully

[2] Starting detector on port 9999...
    Waiting for connection from VM1...
    Connected to traffic generator at 192.168.56.101

[3] Receiving and classifying flows...
    Processed 100 flows...
    Processed 200 flows...
    Processed 300 flows...
    Processed 400 flows...
    Processed 500 flows...
    Processed 600 flows...
    Processed 700 flows...
    Processed 800 flows...
    Processed 900 flows...
    Processed 1000 flows...
    Received END signal from VM1

[4] Classification Latency (pure model prediction):
    Flows in statistics: 990 (first 10 discarded as warm-up)
    Average: 28.7813 ms
    Median: 26.3213 ms
    Min: 13.0095 ms
    Max: 110.6546 ms
    Std dev: 12.8324 ms

[5] End-to-End Latency (arrival to decision):
    Average: 28.8301 ms
    Median: 26.3590 ms
    Min: 13.0346 ms
    Max: 110.6973 ms
    Std dev: 12.8501 ms

    Total session time: 28.91 seconds
    Throughput: 34.6 flows per second

[6] Detection accuracy on streamed flows: 88.40%

[7] Saving results...
    Saved: /home/mathias/ml-ids-project/results/realtime_detection_results.txt
    Latency data saved for visualization

=====
DETECTION COMPLETE!
=====
mathias@IDS-VM2-Detection:~$ █

```

Figure 4.12: Two-VM Real-Time Detection Experiment Output (VM2 Detector)

Note: The console output labels the arrival-to-decision measurement as “End-to-End Latency”. However, timing begins after the flow record arrives at VM2; therefore, this measurement represents detector-side processing latency rather than complete VM1-to-VM2 end-to-end latency.

4.8.2 Real-Time Latency Results

Table 4.7 presents the classification and detector-side processing latency results for the 990 measured flows.

Statistics	Classification Latency (ms)	Detector-Side Processing Latency (ms)
Average	28.78	28.83
Median	26.32	26.36
Minimum	13.01	13.03
Maximum	110.65	110.70
Std. Dev	12.83	12.85

Table 4.7: Two-VM Detector-side Latency Statistics (990 flows, Random Forest)

The median detector-side processing latency was 26.36 milliseconds, meaning a typical flow was classified approximately 26 milliseconds after being received by the detector, supporting the feasibility of near real-time classification under the controlled conditions.

The average classification latency was 28.78 milliseconds, while the average detector-side processing latency was 28.83 milliseconds. The difference of approximately 0.05 milliseconds shows that record parsing and other detector-side processing added relatively little time beyond the classification stage. However, this comparison does not measure the complete network-transmission delay from VM1 because the detector-side timing began after the record arrived at VM2.

The latency distribution (Figure 4.13) is right-skewed, with a large concentration around 14 to 16 ms, a broader concentration between roughly 35 and 45 ms, and a small number of higher-latency observations forming a tail beyond 100 ms. The median (26.36 ms) is lower than the mean (28.83 ms) because the few slower observations raise the mean. Most records were processed quickly, with some variation likely due to virtual-machine scheduling, memory allocation, or background processing, though the specific cause was not investigated.

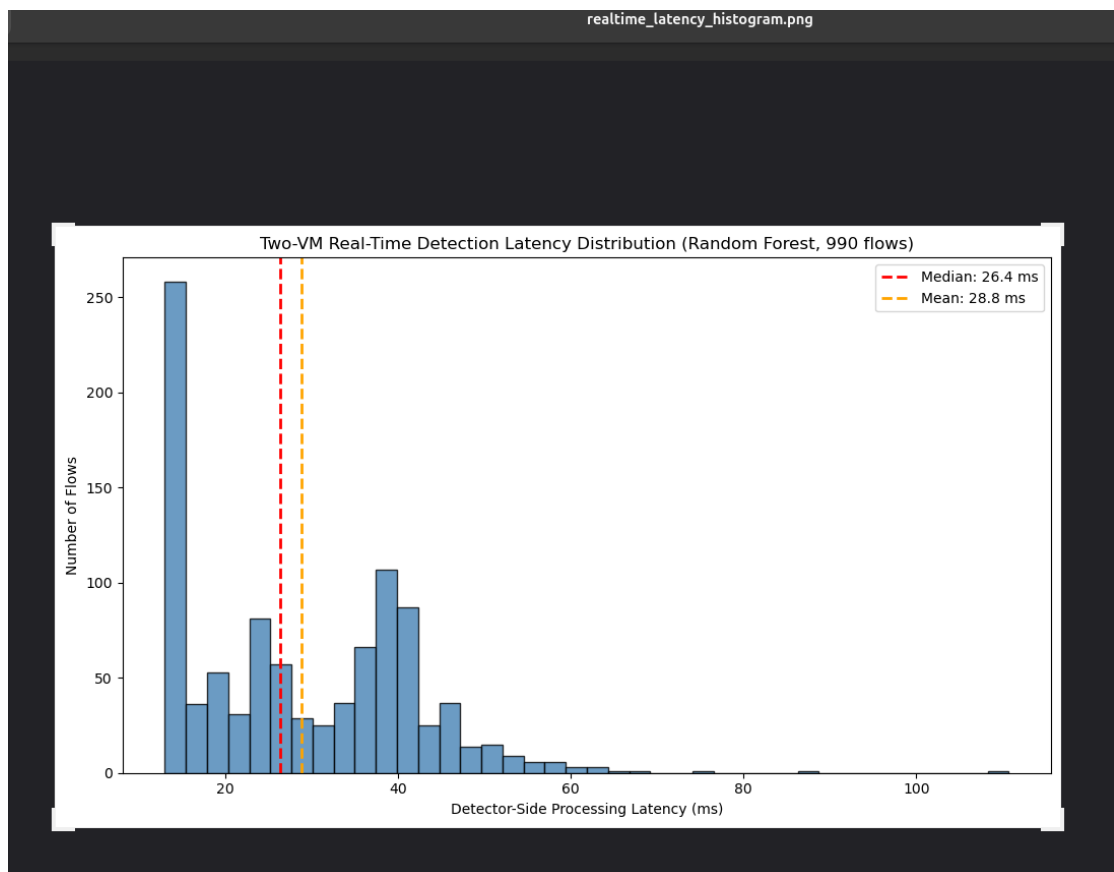


Figure 4.13: Distribution of Detector-Side Processing Latency for 990 flows

4.8.3 Throughput and Detection Accuracy

The sequential, single-threaded system processed approximately 34.6 flows per second. This reflects the measurement design with each flow transmitted and classified individually for accurate per-flow timing and should not be read as the model's maximum capacity; asynchronous processing, multithreading, or small batches could process more, though these were not tested. The detector correctly classified 88.40% of the streamed records, which was close to the Random Forest offline accuracy of 89.75%, with a difference of 1.35 percentage points. This suggests that the model's classification performance remained broadly consistent when records were processed individually. The results were not expected to be identical because the real-time experiment used a smaller sample than the offline evaluation.

Chapter 5: Research Outcomes, Discussion and Evaluation

This chapter brings together and critically evaluates the results presented in Chapter 4 in relation to the research aim and objectives established in Chapter 1. While Chapter

4 focused on the system implementation and the results obtained, this chapter considers what those findings mean when viewed as a whole. It compares the results with existing literature, evaluates the extent to which the research objectives were achieved, and discusses the wider implications and limitations of the study. The discussion is centred on the main research question of whether machine learning-based intrusion detection can operate effectively under near real-time conditions.

5.1 Overview of Key Findings

The study produced two main sets of results: the offline classification performance of the three machine learning models and the near real-time performance of the selected model within the two-virtual-machine environment. Table 5.1 brings together the key findings from both stages of the evaluation.

Finding	Random Forest	Gradient Boosting	SVM
Accuracy (offline)	89.75%	89.86%	87.93%
F1-Score (offline)	91.94%	92.04%	90.44%
ROC AUC (offline)	0.9858	0.9864	0.9593
Training time	4.29 s	46.25 s	800.67 s
Prediction (batch, per sample)	0.0030 ms	0.0017 ms	0.9017 ms
Detector-side median latency	26.36 ms	NOT TESTED	NOT TESTED
Real-time accuracy	88.40%	NOT TESTED	NOT TESTED

Table 5.1: Consolidated Summary of Key Results

Three findings can be drawn from the results. First, all three algorithms performed well during offline evaluation, with accuracy ranging from 87.93% to 89.86% and precision above 98% in every case. Gradient Boosting achieved the highest accuracy and F1-score, although the improvement over Random Forest was small.

Second, the models differed in computational cost. Training time ranged from about four seconds for Random Forest to approximately thirteen minutes for SVM, while prediction times varied by several orders of magnitude. This shows that models with

similar classification performance may still require very different levels of processing time and resources.

Third, Random Forest also performed effectively in the two-virtual-machine experiment. It achieved a median detector-side latency of 26.36 milliseconds and an accuracy of 88.40%, which was close to its offline accuracy of 89.75%. Overall, the findings show that accuracy alone is not enough to determine whether a model is suitable for near real-time intrusion detection. Computational cost, detector-side latency and practical deployment requirements also need to be considered.

5.2 Comparative Performance with the Literature

Random Forest and Gradient Boosting produced closely matched results, and both performed better than SVM. Ahmad and AlQahtani (2023) compared Random Forest, Gradient Boosting and a deep neural network and found Random Forest to be the strongest model across several measures. The present results were slightly different because Gradient Boosting achieved the highest accuracy and F1-score. However, the margin between the two ensemble methods was small, indicating that both were effective for binary intrusion detection.

Halbouni et al. (2022) reported strong intrusion-detection performance using a hybrid CNN-LSTM architecture. That approach involved a more complex neural-network structure, whereas the present investigation examined three supervised machine learning algorithms and considered computational efficiency alongside classification quality. Direct comparison should therefore be approached carefully because results can be affected by differences in model design, preprocessing, feature selection and evaluation procedure.

Dataset selection provides another source of variation. UNSW-NB15 was created partly in response to limitations associated with older benchmark datasets and contains a broader range of recent attack categories. However, it was still generated within a controlled laboratory environment (Moustafa and Slay, 2015).

The number of predictive variables was reduced from 42 to 20 before training. This decision may have affected classification performance, but its exact effect cannot be established because the models were not evaluated separately with the complete 42-feature set. The reduction was made deliberately to limit computational demand and better support the near real-time objective of the system.

Many related investigations concentrate mainly on offline classification. Ahmad and AlQahtani (2023) and Halbouni et al. (2022), for example, reported classification measures without evaluating detector-side latency across a networked environment. The present work included classification performance, training time, prediction time and detector-side latency within one controlled framework. This provided an additional view on whether strong offline results could be maintained when records were transmitted and processed individually (Imanbayev et al., 2022).

5.3 The Accuracy versus Efficiency Trade-off

The results show that accuracy alone provides an incomplete basis for selecting an intrusion detection model. The three algorithms differed by fewer than two percentage points in accuracy, yet training and prediction times varied substantially.

SVM provided the clearest example of this difference. While it achieved a reasonable accuracy of 87.93%, it required 800.67 seconds to train, making it approximately 187 times slower than Random Forest. Its average prediction time of 0.9017 ms per record was also around 300 times slower than Random Forest and more than 500 times slower than Gradient Boosting. The requirements make SVM less suitable for continuously processing large volumes of traffic. The difference between the two ensemble methods was subtler: Gradient Boosting achieved marginally higher accuracy at 89.86%, recall, and F1-score, but its training time (46.25s) was approximately eleven times longer than Random Forest (4.29s), while both prediction times were sufficiently fast that the practical difference was limited. Random Forest therefore provided the stronger balance between classification performance, training efficiency, and prediction speed.

These findings support the argument presented by Halbouni et al. (2022) that models with strong predictive performance may still be unsuitable where computational efficiency and detection speed matter and show that this trade-off also applies to traditional supervised algorithms, not only to more computationally demanding deep-learning methods. The most accurate model is therefore not automatically the most suitable model for near real-time intrusion detection. This was the reason Random Forest was selected for the two-virtual-machine evaluation.

5.4 Near Real-Time Detection Feasibility

The two-virtual-machine experiment provides the clearest answer to the main research question and is one of the study's practical contributions. The literature review identified an important gap in this area. Imanbayev et al. (2022) noted that offline accuracy does not necessarily guarantee effective real-time performance and highlighted feature-extraction latency as a possible limitation. However, their study did not test this through an implemented networked system or compare individual algorithms under time-sensitive conditions. This research partially addresses that gap by designing, implementing and measuring a prototype within a controlled two-virtual-machine environment.

The results indicate that near real-time detection is feasible under the experimental conditions. The Random Forest detector received, prepared and classified each network-flow record with a median detector-side latency of 26.36 milliseconds and this means that a typical flow was classified approximately 26 milliseconds after arriving at VM2. It should be noted that the model was trained and saved before the experiment (VM2 loaded the saved model), and that the existing UNSW-NB15 flow records were used rather than raw packets captured and processed from live traffic.

Two aspects deserve attention. First, the average classification latency (28.78 ms) and detector-side processing latency (28.83 ms) differed by approximately 0.05ms, indicating that record parsing and preparation added little beyond classification stage. The complete end-to-end transmission time from VM1 to VM2 was not measured because timing began after the record arrived at VM2. Second, the real-time accuracy of 88.40% was broadly consistent with the Random Forest offline accuracy of 89.75% (a 1.35 percentage difference). This suggests that the classification performance was largely preserved when records were processed individually rather than in one batch. Nevertheless, the streamed experiment used a smaller sample within a controlled environment, so the result should not be treated as evidence of production performance. Overall, the experiment provides evidence that machine learning-based intrusion detection can achieve near real-time classification under the controlled conditions used in the study.

5.5 Achievement of Research Objectives

This section assesses the extent to which each research objective presented in Chapter 1 was achieved.

Objective 1: critically review the literature on traditional and machine learning-based intrusion detection systems.

This objective was achieved in Chapter 2 through a systematic review of machine learning-based intrusion detection, network traffic datasets and near real-time detection challenges. The review identified the field's strong emphasis on offline accuracy at the expense of response latency, within the principal studies summarised in Table 2.1.

Objective 2: develop a machine learning-based intrusion detection system using Python and the UNSW-NB15 dataset.

This objective was achieved through the implementation in Chapter 4, developed in Python within a controlled two-virtual-machine environment and covering dataset exploration, preprocessing, encoding, normalisation, feature selection, model training, persistence, and a networked detection prototype.

Objective 3: train and evaluate Random Forest, Gradient Boosting and Support Vector Machine models for binary network traffic classification.

This was achieved. All three were trained on the same selected features and evaluated on the predefined test set (Section 4.7). Gradient Boosting achieved the highest classification performance. Random Forest provided the strongest overall balance, and SVM achieved acceptable performance at substantially higher computational cost.

Objective 4: to compare the models using accuracy, precision, recall, F1-score, confusion matrices, training time and prediction time, and to evaluate detector-side response latency for the selected model.

This objective was achieved (Sections 4.7 and 4.8). All three were assessed on classification metrics and timing, and Random Forest was additionally evaluated for detector-side latency, throughput, and streamed accuracy.

Objective 5: To assess the feasibility of the proposed system within a controlled two-virtual-machine environment.

This objective was achieved. Random Forest classified the transmitted records with a median detector-side latency of 26.36 ms at an accuracy of 88.40%, demonstrating feasibility under controlled conditions. As the experiment used existing UNSW-NB15 flow records rather than live packet capture and feature extraction, the findings represent prototype feasibility rather than product readiness.

Overall, all five research objectives were achieved within the defined scope of the study and the overall aim of designing, implementing and evaluating a machine learning-based intrusion detection system capable of detecting malicious network traffic under controlled near real-time conditions was therefore achieved. The limitations of the dataset and experimental environment must be considered when interpreting the wider applicability of the findings.

5.6 Limitations of the Research

Although the research achieved the stated objectives, there are several limitations that must be considered when interpreting the findings.

Dataset limitations: Only the UNSW-NB15 dataset was used. Although widely used and contains relatively modern, single-dataset evaluation limits the generalisability of the findings. A model that performs well on UNSW-NB15 may perform differently when evaluated using another dataset or operational network traffic. Cross-dataset evaluation would therefore provide stronger evidence of the generalisability of the results.

Binary classification scope: The study was restricted to binary classification, distinguishing between normal and attack traffic, rather than identifying individual attack types. As discussed in Section 4.3, this decision was influenced by the severe imbalance in the distribution of the different attack categories. However, it limits the practical usefulness of the system because it identifies that an attack has occurred without determining the specific type of attack.

Real-time evaluation constraints: The experiment used preprocessed flow records rather than capturing live packets and extracting features in real time. A complete operational system would need to capture raw network packets, reconstruct network flows and calculate the selected features before classification. These additional stages would introduce processing time that was not measured in this study. Both virtual machines also operated on the same physical host and communicated through a host-only network. Therefore, the environment did not reproduce the complexity of physically separate systems connected through an operational network. In addition, the flow records were transmitted sequentially under controlled laboratory conditions rather than under bursty or high-volume traffic conditions. The experiment consequently demonstrates near real-time feasibility under controlled conditions rather than production-level performance.

Model and feature optimisation: The algorithms were configured using fixed parameters rather than an extensive hyperparameter-optimisation process. Alternative values for parameters such as the number and depth of trees, learning rate, C and gamma could produce different classification results. In addition, the performance of the selected 20 features was not directly compared with performance using all 42 predictive features. Therefore, the precise effect of feature selection on accuracy and computational efficiency cannot be confirmed.

Single algorithm in real-time testing: Only Random Forest was evaluated within the two-virtual-machine experiment. Although Gradient Boosting achieved slightly stronger offline classification results and faster batch prediction, its detector-side latency and throughput were not measured. A direct near real-time comparison between the two ensemble models was therefore not possible.

These limitations do not invalidate the findings, but they restrict the extent to which the results can be generalised beyond the dataset and controlled experimental environment used in this research.

5.7 Implications and Future Work

The results have implications for both model evaluation and practical system design. They also point to several areas in which the prototype could be extended and tested further.

Theoretical Implications

The findings support the argument by Imanbayev et al. (2022) that response latency should be considered alongside classification measures when evaluating machine learning-based intrusion-detection systems. Models with similar accuracy did not necessarily require similar amounts of training or prediction time. Consequently, accuracy by itself did not provide enough information to determine whether a model would be suitable for operational deployment.

The comparison also showed the importance of considering computational demand together with classification performance. Gradient Boosting achieved the strongest accuracy, whereas Random Forest offered the more favourable overall balance of accuracy, training time and prediction speed. SVM produced reasonable classification results but required considerably more processing time. The highest-

scoring model was therefore not automatically the most practical choice for near real-time intrusion detection.

Practical Implications

The results offer a practical basis for selecting models for machine learning-based intrusion detection. Random Forest and Gradient Boosting both achieved strong classification results and short batch-prediction times. Random Forest was chosen for the prototype because its performance remained close to that of Gradient Boosting while requiring substantially less time for training.

During the two-virtual-machine evaluation, the Random Forest detector processed individually transmitted flow records with a median detector-side latency of 26.36 milliseconds. This shows that the prototype responded within a short period under the conditions used in the experiment. However, the inputs had already been preprocessed, and the evaluation did not include live packet capture or real-time feature extraction. The result therefore demonstrates prototype feasibility rather than readiness for production deployment.

Future Work

Further work could extend the prototype by introducing live packet capture and feature extraction so that latency could be measured across the complete detection process. Gradient Boosting could also be evaluated in the same two-virtual-machine environment to establish whether its slightly stronger offline results would remain consistent when records were processed one at a time. Testing with CICIDS2017, CSE-CIC-IDS2018 or other datasets would provide further evidence of whether the findings extend beyond UNSW-NB15.

A multiclass version of the system could be developed to distinguish between individual attack categories. Class weighting, oversampling and feature-selection methods aimed at minority classes could then be examined to address the uneven number of records in each category.

Testing on separate physical machines would reduce the effect of shared host resources. Additional evaluation under high-volume or bursty traffic would also provide a more realistic view of latency, throughput and stability. Other possible extensions include hyperparameter optimisation, asynchronous processing, multithreading and controlled batch prediction. Alert generation and response logging

could then be added as the prototype develops into a more complete intrusion-detection system.

5.8 Summary

This chapter examined the main findings in relation to both classification performance and computational efficiency. All three algorithms produced strong binary-classification results on UNSW-NB15, although their computational requirements were not equally favourable.

Random Forest was selected for the two-virtual-machine evaluation because it provided the strongest overall balance between classification performance and computational cost. In the networked experiment, it achieved a streamed accuracy of 88.40% and recorded a median detector-side latency of 26.36 milliseconds. These findings support the feasibility of near real-time machine learning classification under controlled conditions and help address the response-latency gap identified in the literature.

All five research objectives were achieved within the defined scope of the study. However, the use of a single dataset, preprocessed flow records, binary classification and a controlled virtual-machine environment limits the extent to which the findings can be generalised. The next chapter discusses how the research project was managed and carried out.

Chapter 6: Project Management

This chapter explains how the research project was managed, planning through to implementation. It covers time management, communication with the supervisor, the challenges encountered and the steps taken to resolve them. The chapter also discusses changes made to the project scope and reflects on the project management approach and the lessons learned during the research.

6.1 Project Planning and Scheduling

The project was divided into several stages to ensure that the research could be completed within the available dissertation period. These stages included refining the topic, reviewing the literature review, developing the methodology, setting up the environment, preprocessing the dataset, selecting features, training the models, evaluating the system and writing the dissertation.

At the start of the project, the proposed topic covered machine learning, Software-Defined Networking and micro-segmentation. The scope was later narrowed to the design and implementation of a machine learning-based intrusion detection system for near real-time network traffic analysis. This made the project more manageable within the available timeframe and allowed more attention to be given to model comparison and response latency evaluation.

The implementation tasks were completed in a logical sequence because each stage relied on the one before it. The virtual-machine environment was configured first, followed by dataset exploration and preprocessing. Feature selection and model training were then completed before the offline results were evaluated. Based on these results, Random Forest was chosen for the two-virtual-machine experiment.

The dissertation was written alongside the technical work rather than being left until the end of the project. The methodology and implementation chapters were updated as the system developed, and new experimental results became available. This helped ensure the written discussion remained consistent with the final implemented system.

6.2 Project Execution and Time Management

The project required time to be divided between technical development and academic writing. The technical activities included configuring the two virtual machines, installing the required software, processing a large network dataset, training three machine-learning models and evaluating the selected model in a networked environment.

Work was completed in stages, and the output from each stage was checked before the next stage began. Dataset exploration was completed before preprocessing, the processed data were verified before feature selection, and the selected features were checked before model training. This sequence made it easier to identify problems early and reduced the likelihood of errors affecting later stages of the implementation.

Some activities required more time than originally expected. These included restructuring the literature review, aligning the dissertation with the implemented system, formatting figures and tables, and refining the descriptions of preprocessing and latency measurement. Additional time was therefore given to revising earlier chapters while the later implementation work continued.

Separate Python scripts were used for dataset exploration, preprocessing, feature selection, model training and near real-time evaluation. This allowed each component to be developed and tested independently. The use of separate directories for datasets, models, scripts and results also made the project files easier to organise and reduced the risk of inconsistent experimental outputs being used.

6.3 Supervisor Engagement and Feedback

Supervisor engagement contributed to the refinement of both the research scope and the dissertation structure. Meetings were used to review progress, discuss the direction of the research, identify areas requiring improvement and agree the next stages of the project.

Early feedback showed that the original proposal and research poster covered too many technical areas. The initial scope combined machine learning-based intrusion detection, Software-Defined Networking and micro-segmentation. Following this feedback, the project was narrowed to supervised machine learning-based intrusion detection under near real-time conditions. The reduced scope was more achievable within the available period and allowed greater attention to be given to implementation and evaluation.

Feedback also led to changes in the presentation of the literature review and methodology. The literature-review process was expanded to describe the databases searched, the keywords used, the inclusion and exclusion criteria and the method used to select relevant studies. A literature-summary table was also introduced to make the research gaps identified from previous studies clearer.

The dissertation structure was further improved following additional feedback. A document structure was added to Chapter 1, while implementation-related content was moved from the methodology chapter to Chapter 4. The section on ethical considerations and the software tools used during the research were also explained to improve clarity.

Rather than leaving revisions until the end of the project, supervisor feedback was incorporated throughout the research. This helped maintain consistency between the research objectives, methodology, implementation and evaluation. Records of the meetings and feedback were retained and are included in Appendix C.

6.4 Challenges and Scope Changes

Several challenges were encountered during the project and required adjustments to the original plan.

The first challenge was the breadth of the initial project scope. Combining machine learning, Software-Defined Networking and micro-segmentation would have required the development and evaluation of several major technical components. Completing all these areas within the dissertation period would have limited the depth of the investigation. The scope was therefore reduced to the implementation and evaluation of a machine learning-based intrusion detection system using a controlled two-virtual-machine environment.

A second challenge involved maintaining consistency between the implemented system and the written dissertation. Earlier descriptions referred to live packet capture, feature extraction and complete end-to-end network latency. However, the final experiment transmitted existing UNSW-NB15 flow records and measured processing time after the records reached the detector. The written account was therefore reviewed and revised to ensure that it accurately described the implemented system and did not extend beyond what had been tested.

The algorithms also differed considerably in their computational requirements. SVM took much longer to train and produce predictions than Random Forest and Gradient Boosting. It was retained in the comparison because this difference was itself relevant to the research. Its slower processing demonstrated that similar classification results do not necessarily imply similar computational suitability.

Document formatting created a separate set of difficulties. These included maintaining heading numbering, updating the table of contents, generating the lists of figures and tables, and keeping captions and cross-references accurate. Changes made in one chapter often affected numbering and page references elsewhere, so these elements required repeated checking.

These challenges were managed by retaining focus on the central research objectives, completing the implementation in separate stages and verifying each output before continuing. Although some aspects of the original scope were removed, the central aim of designing and evaluating a machine learning-based intrusion-detection system under controlled near real-time conditions remained unchanged.

6.5 Reflection and Lessons Learned

The project provided useful lessons in research planning, technical implementation and academic writing.

A major lesson concerned the need to define a realistic scope at the beginning of a project. The original proposal included machine learning-based intrusion detection, Software-Defined Networking and micro-segmentation. Although the topics were related, implementing and evaluating all three would have made the investigation too broad. Reducing the scope allowed the work to concentrate on the machine-learning system and its near real-time evaluation. This resulted in a more focused and detailed investigation.

Another lesson concerned the value of completing technical tasks as separate, testable stages. The scripts used for exploration, preprocessing, feature selection and model training were checked individually before the complete detection process was assembled. This made errors easier to locate and correct. Saving intermediate datasets and trained models also avoided unnecessary repetition and helped maintain reproducibility.

The project also showed how important it is for the dissertation to accurately reflect the system that was actually implemented and evaluated. However, the implemented experiment transmitted existing flow records and measured latency after they arrived at the detector. Reviewing these descriptions strengthened the accuracy and credibility of the dissertation.

Another lesson concerned the relationship between classification performance and computational efficiency. Before implementation, model accuracy appeared to be the principal factor for comparison. However, the experiments showed that algorithms with similar accuracy could have substantially different training and prediction times. This improved the understanding of why practical cybersecurity systems must be evaluated using more than classification metrics alone.

Regular supervisor feedback was also valuable. Feedback helped identify weaknesses in the original scope, literature-review structure, methodology and organisation of the dissertation. Acting on the feedback progressively reduced the amount of major correction required at the final stage.

Finally, greater time could have been allocated to document formatting, cross-referencing and the early development of the real-time experiment. Although the technical implementation was completed successfully, preparing figures, updating the table of contents and maintaining consistent terminology required more time than initially expected. Future projects would benefit from using a detailed schedule that includes specific time for technical verification, writing, formatting and final quality assurance.

6.6 Summary

This chapter discussed the management of the research project, including project planning, technical execution, time management, supervisor engagement and the challenges encountered. The project was divided into manageable stages, allowing each component of the intrusion detection pipeline to be developed and verified before integration.

The original scope was narrowed to ensure that the project remained achievable within the available time. Supervisor feedback supported this process and also improved the literature review, methodology and dissertation structure. Challenges involving computational performance, accuracy of the implementation description and document formatting were managed through staged development, repeated verification and continued revision of the dissertation.

The project also demonstrated the importance of setting a realistic scope, reviewing progress throughout the implementation and ensuring that the written dissertation remained aligned with the system that was actually developed. These lessons supported completion of the research aim and objectives.

The following chapter concludes the dissertation by summarising the research, presenting its main contributions and identifying areas for future work.

Chapter 7: Conclusion and Future Work

This chapter concludes the dissertation by summarising the research, restating the main contributions, confirming the achievement of the aim and objectives, critically evaluating the work, and identifying directions for future research.

7.1 Summary of the Research

The aim of this research was to develop and evaluate a machine learning-based intrusion detection system capable of detecting malicious network activity under controlled near real-time conditions. The need arose from a gap in the literature: although many studies report strong classification accuracy for machine learning-based intrusion detection, most evaluate their models only through offline testing on benchmark datasets, and comparatively few consider whether these models can process traffic quickly enough to support timely investigation and response.

To address this gap, a complete pipeline was designed and implemented within a two-virtual-machine environment that separated traffic generation from detection. The UNSW-NB15 dataset was selected for its contemporary attack categories. Following exploration and preprocessing (categorical encoding, feature normalisation, and removal of non-informative variables), Random Forest feature importance was used to reduce the predictive set from 42 to the 20 highest-ranked features. Three supervised algorithms: Random Forest, Gradient Boosting, and Support Vector Machine, were then trained and compared for binary classification using accuracy, precision, recall, F1-score, ROC AUC, confusion matrices, training time, and prediction time. Gradient Boosting achieved the highest accuracy (89.86%), with Random Forest closely comparable (89.75%) at substantially shorter training time. Random Forest was therefore selected for the two-virtual-machine evaluation, where flow records were transmitted individually from VM1 to VM2, achieving a median detector-side latency of 26.36 milliseconds and a streamed accuracy of 88.40%. Because the prototype used existing flow records rather than live packet capture and feature extraction, the results demonstrate prototype feasibility rather than production readiness.

7.2 Key Contributions

The research makes three principal contributions.

The first is a comparison of the classification performance and computational efficiency of three supervised machine-learning algorithms for intrusion detection. Although the

recorded accuracy values were close, ranging from 87.93% to 89.86%, the computational demands of the models were markedly different.

Random Forest completed training in approximately four seconds, whereas SVM required more than 13 minutes. Their prediction times also differed by approximately three orders of magnitude. These results show that models with similar classification accuracy may still differ substantially in processing cost. Computational efficiency must therefore be considered alongside accuracy when selecting a model for near real-time intrusion detection.

Second, the study provides an empirical evaluation of machine learning-based intrusion detection within a controlled two-virtual-machine environment. Random Forest classified individually transmitted UNSW-NB15 flow records with a median detector-side latency of 26.36 milliseconds and achieved a streamed accuracy of 88.40%. These results show that near real-time machine learning classification was feasible under the controlled experimental conditions used in this study.

Third, it presents a practical experimental framework for evaluating intrusion detection models using both offline and networked testing. The methodology combines dataset preprocessing, feature selection, comparative model evaluation, model persistence and individual flow-record transmission between two virtual machines. This approach allows classification performance, computational efficiency and detector-side latency to be compared within the same study and the framework could be adapted in future research to evaluate other algorithms, datasets and intrusion detection architectures.

Collectively, these contributions extend the evaluation of machine learning-based intrusion detection beyond offline accuracy by demonstrating the importance of response latency and computational efficiency for near real-time operation.

7.3 Achievement of aim and objectives

The aim was to develop and evaluate a machine learning-based intrusion detection system that uses network traffic behaviour to detect malicious or abnormal activity in near real-time conditions. This aim was achieved. As presented in Chapter 5, the five supporting objectives of this research were all achieved: a summary was provided of existing intrusion detection techniques and limitations identified by using a systematic literature review; a machine learning-based intrusion detection system was designed and implemented in Python; three machine learning algorithms were trained and

compared using the UNSW-NB15 network traffic dataset; system performance was evaluated using the accuracy, precision, recall, F1-score and detection latency metrics; near real-time detection capability was demonstrated by using two virtual machines.

7.4 Critical evaluation

The research achieved its aim, but a critical evaluation is necessary to assess both the strengths of the work and the limitations affecting interpretation.

One of the main strengths of the research is that it produced a functioning prototype rather than remaining at the level of system design. The work covered the complete experimental sequence, including environment configuration, dataset preprocessing, feature selection, model training and networked detection testing. A consistent comparison was maintained by using the same dataset, selected features and evaluation measures for all three algorithms.

A further strength is the inclusion of computational efficiency and detector-side latency alongside classification performance. Many intrusion-detection studies emphasise accuracy-based measures, whereas this research also examined whether the models could support near real-time operation. The model-selection decision reflected this wider evaluation. Although Gradient Boosting returned the highest offline accuracy, Random Forest was selected for the networked test because it provided a more suitable balance of classification quality, training time and prediction speed.

However, the limitations discussed in Chapter 5 restrict wider interpretation of the results. The near real-time experiment used preprocessed flow records rather than live packet capture and real-time feature extraction. The measured latency therefore represented detector-side processing only and did not cover the complete process from raw packet capture to alert generation. Both virtual machines also ran on the same physical host and communicated over a host-only network, which did not reproduce the delays or distribution of resources that would occur between separate physical machines.

In addition, 1,000 records were transmitted sequentially, rather than being tested under sustained high-volume or bursty traffic. The use of a single dataset and binary classification also limits generalisability and prevents the system from identifying specific attack categories. Furthermore, only Random Forest was tested in the two-

virtual-machine environment, so no direct networked comparison was made with Gradient Boosting. Overall, the findings provide evidence that the prototype was feasible under controlled conditions rather than demonstrating production readiness. The study therefore provides a foundation for further evaluation using live network traffic, real-time feature extraction, physically separate machines and more demanding traffic conditions.

7.5 Future Work

Future work should focus on developing the prototype into a more complete intrusion detection pipeline. The first priority would be to integrate live packet capture and real-time feature extraction. This would allow raw packets to be collected, grouped into flows and converted into the features required by the model, while also measuring the additional latency introduced by these stages.

Gradient Boosting should also be tested within the existing two-virtual-machine environment to allow a direct near real-time comparison with Random Forest. SVM could be reconsidered after hyperparameter optimisation, although its higher computational cost makes it a lower priority. Evaluation using other datasets, such as CICIDS2017 or CSE-CIC-IDS2018, would provide stronger evidence of whether the findings generalise beyond UNSW-NB15.

The system could also be extended from binary to multiclass classification so that specific attack categories can be identified. This would require techniques such as class weighting or oversampling to reduce the effect of class imbalance.

Testing the traffic generator and detector on separate physical machines, under different traffic volumes and burst conditions, would provide a more realistic assessment of throughput, latency and system stability. Further work could also include systematic hyperparameter tuning, a direct comparison of the 20-feature and 42-feature datasets, and throughput optimisation through asynchronous processing, multithreading and controlled batch prediction. Alert generation, event logging and an analyst interface could then be added to develop a more complete intrusion detection workflow.

7.6 Concluding Remarks

This research designed, implemented and evaluated a machine learning-based intrusion detection system under controlled near real-time conditions. Although

Gradient Boosting achieved the highest classification accuracy, Random Forest provided the strongest overall balance between accuracy, training time and prediction speed. During the two-virtual-machine experiment, Random Forest classified individually transmitted flow records with an accuracy of 88.40% and a median detector-side latency of 26.36 milliseconds.

The findings show that machine learning-based classification can support near real-time intrusion detection within a controlled experimental environment. They also demonstrate that classification accuracy should not be considered in isolation when selecting an algorithm. A suitable model must combine reliable detection performance with low computational cost and fast response time.

The study does not establish production readiness because live packet capture, real-time feature extraction and high-volume operational traffic were outside its scope. However, the implemented prototype and experimental framework provide a practical basis for future research into more complete and scalable machine learning-based intrusion detection systems.

References

- Ahmad, I. and AlQahtani, H.S. (2023) 'A comparative analysis of Gradient Boosting, Random Forest and Deep Neural Networks in intrusion detection system', *ARPJN Journal of Engineering and Applied Sciences*, 18(12), pp. 1392–1402.
- Alhakeem, M.S. and Bin Ajlan, K. (2024) 'A comparative evaluation of machine learning-based intrusion detection systems for securing cloud environments', *Journal of Information Security and Cybercrimes Research*, 7(2), pp. 127–142. doi: 10.26735/RSND3740.
- Great Britain (1990) *Computer Misuse Act 1990*. London: The Stationery Office.
- Halbouni, A.H., Gunawan, T.S., Habaebi, M.H., Halbouni, M., Kartiwi, M. and Ahmad, R. (2022) 'CNN-LSTM: Hybrid deep neural network for network intrusion detection system', *IEEE Access*, 10, pp. 99837–99849. doi: 10.1109/ACCESS.2022.3206425.
- Imanbayev, A., Tynymbayev, S., Odarchenko, R., Gnatyuk, S., Berdibayev, R., Baikenov, A. and Kaniyeva, N. (2022) 'Research of machine learning algorithms for the development of intrusion detection systems in 5G mobile networks and beyond', *Sensors*, 22(24), article 9957. doi: 10.3390/s22249957.
- Moustafa, N. and Slay, J. (2015) 'UNSW-NB15: A comprehensive data set for network intrusion detection systems', in *2015 Military Communications and Information Systems Conference*. Canberra, Australia: IEEE, pp. 1–6. doi: 10.1109/MILCIS.2015.7348942.
- Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- vom Brocke, J., Hevner, A. and Maedche, A. (eds.) (2020) *Design Science Research: Cases*. Cham: Springer. doi: 10.1007/978-3-030-46781-4.
- Yeoh, W., Liu, M., Shore, M. and Jiang, F. (2023) 'Zero trust cybersecurity: Critical success factors and a maturity assessment framework', *Computers & Security*, 133, article 103412.

Appendices

Appendix A

Originality and Use of Generative Artificial Intelligence (GAI) Statement

- ✓ I understand that to use the work and ideas of others, including generative AI output, without full acknowledgement, is academic unfair practice.
- ✓ I confirm that this coursework submission is all my own, original work and that all sources, summaries, paraphrases, and quotes are fully referenced as required by the LBU Academic Regulations.

DECLARATION OF GENERATIVE AI USE:

I DID use Generative AI technology in the development, writing, or editing of this assignment. I have included a copy of the reference used and the output generated in the Appendix section

Numbering	Generative AI Tool (e.g. ChatGPT)	How generative AI Tool was used	Reference
1	ChatGPT 5.6	Used to review and refine sections of the dissertation for academic clarity, grammar, structure and consistency. Suggestions were reviewed and adapted	OpenAI. (2026). ChatGPT [Large language model]. https://chatgpt.com/share/6a89a9db-e3a8-83eb-a92d-2cc71b7085e9

		before inclusion.	
2	ChatGPT 5.5	Used to review the final dissertation for grammar, consistency in terminology and reported results, and areas that required clarification.	OpenAI. (2026). ChatGPT [Large language model]. https://chatgpt.com/share/6a89b7c5-c2e4-83eb-8fae-c77a9543529c
3	Grammarly	Used to check spelling, grammar, punctuation and sentence clarity during proofreading. Suggested corrections were reviewed	Grammarly. (2026). Grammarly [AI-assisted writing tool].

		before being accepted.	
--	--	---------------------------	--

Appendix B

Ethics Approval

Ethics approval screenshot from ethics application submission system.

Worktribe

Search

Mathias Amade

[Home](#) [Messages](#) [Profiles](#) [Projects](#) [Ethics](#) [Help](#)

Hello Mathias Amade

Welcome to Worktribe.

Recent

Ethics Applications

Ethics Applications Assigned To Me (1)



Design and Implementation of a Real-Time Machine Learning Based Intrusion Detection System for Network Traffic Analysis

Risk

Low

Applicant

Mathias Amade

Org Unit

School of Built Environment, Engineering and Computing

Supervisor

Mo Hassan

Status

Favourable Opinion

My Ethics Applications (1)



Design and Implementation of a Real-Time Machine Learning Based Intrusion Detection System for Network Traffic Analysis

Risk

Low

Applicant

Mathias Amade

Org Unit

School of Built Environment, Engineering and Computing

Supervisor

Mo Hassan

Status

Favourable Opinion

Appendix C: Evidence of Meetings with Supervisors

MSc Dissertation - Progress update  Summarize this email

 Hassan, Mo

 To:  Amade, Mathias (Student)

  Reply  Reply all  Forward  

Fri 2026-06-26 10:16

 You replied on Fri 2026-06-26 11:28

Good morning, Mathias,

I hope you are well.

Could you please update me on your research/project progress?

All the best,

Mo

 Amade, Mathias (Student)

 To:  Hassan, Mo

Fri 2026-06-26 11:28

 MSc Cyber Security Dissertati...
801 KB

Dear Mo,

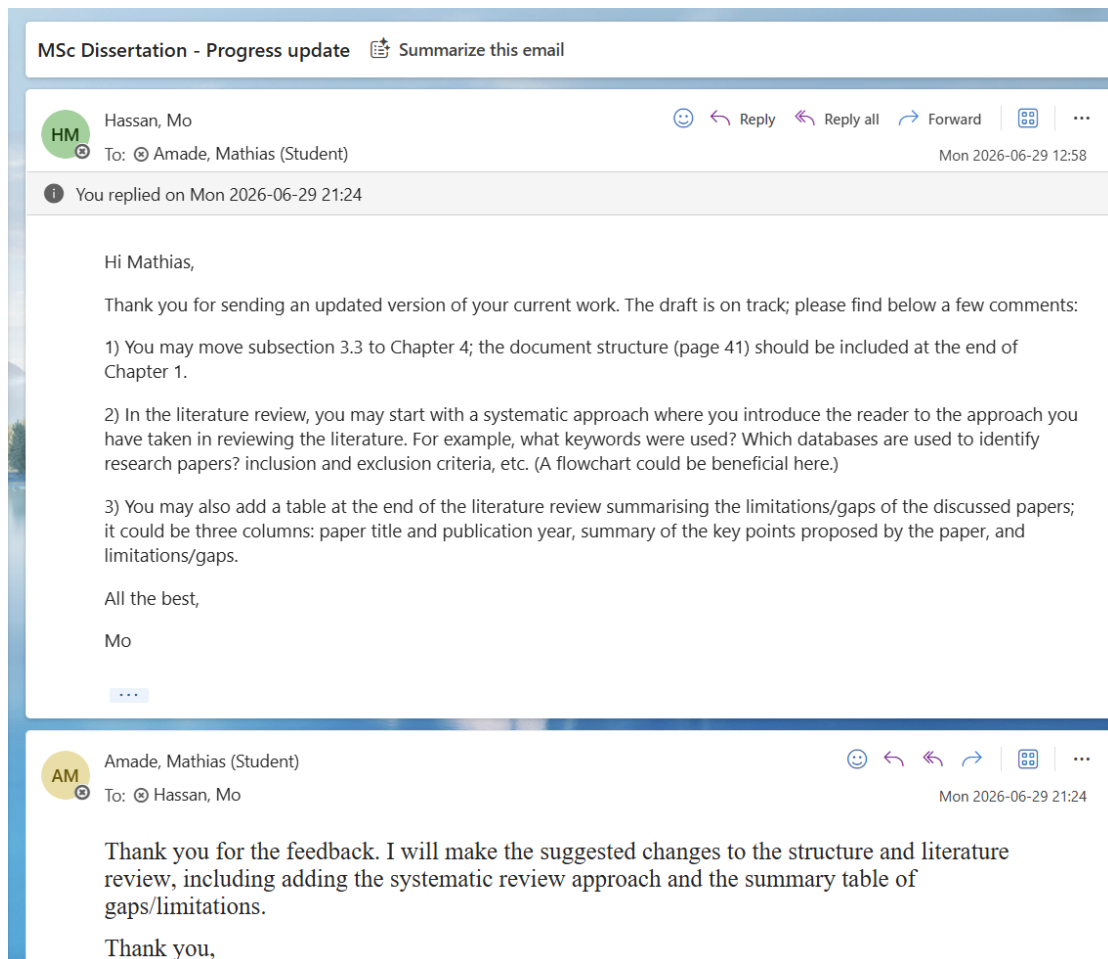
Good morning. I hope you are well too.

Thank you for your email.

I have made the corrections you suggested and continued working on my project. I have also made further progress on Chapter 4, especially with the implementation, results, and analysis sections.

Please find attached my updated Chapters 1–4 for your review. I would be grateful for any feedback or guidance on areas that may still need improvement.

Thank you for your support.



Appendix D: Implementation Code

This appendix contains the complete Python source code developed for the intrusion detection system, together with the command-line steps used to configure the environment. All scripts were written and executed within the two-virtual-machine environment described in Chapter 4. The scripts are presented in the order in which they were used in the implementation pipeline: environment setup, preprocessing, feature selection, model training (Random Forest, Gradient Boosting, Support Vector Machine), and the two-virtual-machine real-time detection experiment (traffic generator and detector).

D.0 Environment Setup Commands

The following commands were used to configure the software environment on both virtual machines (Ubuntu 26.04 LTS).

System update

```
sudo apt update && sudo apt upgrade -y
```

Development tools

```
sudo apt install -y python3-pip python3-dev build-essential
```

Scientific libraries (installed from the OS repository for Python 3.14 compatibility)

```
sudo apt install -y python3-numpy python3-pandas python3-sklearn \
python3-matplotlib python3-seaborn
```

Packet manipulation and progress monitoring (via pip)

```
pip3 install --break-system-packages scapy tqdm
```

The following commands were used on VM2 to enable secure file transfer and to receive the trained model from VM1 for the two-virtual-machine experiment.

On VM2: install and enable the SSH server (required for SCP transfer)

```
sudo apt install -y openssh-server
```

```
sudo systemctl start ssh
```

```
sudo systemctl enable ssh
```

Create the project directory structure on VM2

```
mkdir -p ~/ml-ids-project/models ~/ml-ids-project/data \
```

```
~/ml-ids-project/results ~/ml-ids-project/scripts
```

```
# On VM1: transfer the trained model to VM2 over the host-only network
```

```
scp ~/ml-ids-project/models/random_forest_model.pkl \  
mathias@192.168.56.102:~/ml-ids-project/models/
```

```
# Verify VM-to-VM connectivity before running the experiment
```

```
ping -c 3 192.168.56.102
```

D.1 Data Preprocessing (`02\_preprocess\_data.py`)

Runs on VM1. Loads the UNSW-NB15 dataset, removes non-informative columns, encodes categorical features, standardises numerical features, and saves the preprocessed training and testing sets.

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
DATA_PATH = '/home/mathias/ml-ids-project/data/'
```

```
TRAINING_FILE = DATA_PATH + 'UNSW_NB15_training-set.csv'
```

```
TESTING_FILE = DATA_PATH + 'UNSW_NB15_testing-set.csv'
```

```
TRAIN_OUT = DATA_PATH + 'train_preprocessed.csv'
```

```
TEST_OUT = DATA_PATH + 'test_preprocessed.csv'
```

```
# Load data
```

```
train_df = pd.read_csv(TRAINING_FILE)
```

```

test_df = pd.read_csv(TESTING_FILE)

# Identify categorical columns
categorical_cols = ['proto', 'service', 'state']

# Drop non-informative columns (id = record identifier, attack_cat = leakage)
cols_to_drop = ['id', 'attack_cat']
train_df = train_df.drop(cols_to_drop, axis=1)
test_df = test_df.drop(cols_to_drop, axis=1)

# Encode categorical features (fit on combined data for consistent mapping)
for col in categorical_cols:
    encoder = LabelEncoder()
    combined = pd.concat([train_df[col], test_df[col]], axis=0)
    encoder.fit(combined)
    train_df[col] = encoder.transform(train_df[col])
    test_df[col] = encoder.transform(test_df[col])

# Separate features and labels
X_train = train_df.drop('label', axis=1)
y_train = train_df['label']
X_test = test_df.drop('label', axis=1)
y_test = test_df['label']

# Normalise features (fit on training data only to avoid leakage)

```



```

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


# Recombine and save

X_train_final = pd.DataFrame(X_train_scaled, columns=X_train.columns)

X_test_final = pd.DataFrame(X_test_scaled, columns=X_test.columns)

X_train_final['label'] = y_train.values

X_test_final['label'] = y_test.values

X_train_final.to_csv(TRAIN_OUT, index=False)

X_test_final.to_csv(TEST_OUT, index=False)

```

D.2 Feature Selection (`03\_feature\_selection.py`)

Runs on VM1. Trains a Random Forest to compute feature importance, selects the top 20 features, saves the reduced datasets, and generates the feature-importance chart.

```

import pandas as pd

import numpy as np

from sklearn.ensemble import RandomForestClassifier

import matplotlib

matplotlib.use('Agg')

import matplotlib.pyplot as plt


DATA_PATH = '/home/mathias/ml-ids-project/data/'

RESULTS_PATH = '/home/mathias/ml-ids-project/results/'

TRAIN_FILE = DATA_PATH + 'train_preprocessed.csv'

```

```

TEST_FILE = DATA_PATH + 'test_preprocessed.csv'

TOP_N_FEATURES = 20

# Load preprocessed data

train_df = pd.read_csv(TRAIN_FILE)

test_df = pd.read_csv(TEST_FILE)

X_train = train_df.drop('label', axis=1)

y_train = train_df['label']

X_test = test_df.drop('label', axis=1)

y_test = test_df['label']

# Train Random Forest for feature importance

rf_selector = RandomForestClassifier(

    n_estimators=100, max_depth=15, random_state=42, n_jobs=-1

)

rf_selector.fit(X_train, y_train)

# Rank features by importance

feature_importance = pd.DataFrame({

    'feature': X_train.columns,

    'importance': rf_selector.feature_importances_

}).sort_values('importance', ascending=False)

# Select top N features
90

```

```

top_features = feature_importance.head(TOP_N_FEATURES)['feature'].tolist()

# Create and save reduced datasets

X_train_selected = X_train[top_features].copy()

X_test_selected = X_test[top_features].copy()

X_train_selected['label'] = y_train.values

X_test_selected['label'] = y_test.values

X_train_selected.to_csv(DATA_PATH + 'train_selected.csv', index=False)

X_test_selected.to_csv(DATA_PATH + 'test_selected.csv', index=False)

feature_importance.to_csv(RESULTS_PATH + 'feature_importance.csv',
index=False)

# Feature importance chart (top 20)

plt.figure(figsize=(10, 8))

top20 = feature_importance.head(20)

plt.barh(range(len(top20)), top20['importance'], color='steelblue')

plt.yticks(range(len(top20)), top20['feature'])

plt.xlabel('Feature Importance Score')

plt.title('Top 20 Most Important Features (Random Forest)')

plt.gca().invert_yaxis()

plt.tight_layout()

plt.savefig(RESULTS_PATH + 'feature_importance_chart.png',
bbox_inches='tight', dpi=100,

```

D.3 Random Forest Training (`04\_train\_random\_forest.py`)

Runs on VM1. Trains and evaluates the Random Forest classifier, producing metrics, a confusion matrix, and the saved model.

```
import pandas as pd

import numpy as np

import time

import joblib

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report,
                             roc_auc_score)

import matplotlib

matplotlib.use('Agg')

import matplotlib.pyplot as plt

import seaborn as sns


DATA_PATH = '/home/mathias/ml-ids-project/data/'

MODELS_PATH = '/home/mathias/ml-ids-project/models/'

RESULTS_PATH = '/home/mathias/ml-ids-project/results/'


train_df = pd.read_csv(DATA_PATH + 'train_selected.csv')

test_df = pd.read_csv(DATA_PATH + 'test_selected.csv')

X_train = train_df.drop('label', axis=1)

y_train = train_df['label']

X_test = test_df.drop('label', axis=1)
```

```

y_test = test_df['label']

# Configure and train
rf_model = RandomForestClassifier(
    n_estimators=100, max_depth=20, min_samples_split=5,
    min_samples_leaf=2, max_features='sqrt', random_state=42, n_jobs=-1
)

train_start = time.time()

rf_model.fit(X_train, y_train)

train_time = time.time() - train_start

# Predict and time
predict_start = time.time()

y_pred = rf_model.predict(X_test)

predict_time = time.time() - predict_start

per_sample_time = (predict_time / len(X_test)) * 1000

# Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

y_pred_proba = rf_model.predict_proba(X_test)[:, 1]

auc = roc_auc_score(y_test, y_pred_proba)

cm = confusion_matrix(y_test, y_pred)

```

```

# Confusion matrix chart

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',

            xticklabels=['Normal', 'Attack'], yticklabels=['Normal', 'Attack'])

plt.title('Random Forest - Confusion Matrix')

plt.ylabel('Actual'); plt.xlabel('Predicted')

plt.tight_layout()

plt.savefig(RESULTS_PATH + 'rf_confusion_matrix.png', dpi=100,
            bbox_inches='tight')

# Save model

joblib.dump(rf_model, MODELS_PATH + 'random_forest_model.pkl')

```

Note: The Gradient Boosting script (`05\_train\_gradient\_boosting.py`) and the SVM script (`06\_train\_svm.py`) follow the same structure as above, substituting `GradientBoostingClassifier(n\_estimators=100, learning\_rate=0.1, max\_depth=5, min\_samples\_split=5, min\_samples\_leaf=2, random\_state=42)` and `SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random\_state=42)` respectively for the classifier, and saving to `gradient\_boosting\_model.pkl` and `svm\_model.pkl`.*

D.4 IDS Detector — VM2 (`07\_ids\_detector.py`)

Runs on VM2. Loads the trained model, listens for incoming flow records over a TCP socket, classifies each in real time, and measures classification and detector-side processing latency.

```
import socket
```

```
import json

import time

import warnings

import numpy as np

import joblib


warnings.filterwarnings("ignore")


MODEL_PATH = '/home/mathias/ml-ids-project/models/random_forest_model.pkl'

RESULTS_PATH = '/home/mathias/ml-ids-project/results/'

HOST = '0.0.0.0'

PORT = 9999

WARMUP = 10 # discard first N flows from statistics


model = joblib.load(MODEL_PATH)


server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

server_socket.bind((HOST, PORT))

server_socket.listen(1)

conn, addr = server_socket.accept()


classify_times, total_times, predictions, actuals = [], [], [], []

flow_count = 0

buffer = ""

95
```

```

start_session = time.time()

try:
    while True:
        data = conn.recv(4096).decode('utf-8')

        if not data:
            break

        buffer += data

        while "\n" in buffer:
            line, buffer = buffer.split("\n", 1)

            if line.strip() == "END":
                raise StopIteration

            if not line.strip():
                continue

            arrival_time = time.time()

            record = json.loads(line)

            features = np.array(record['features']).reshape(1, -1)

            actual_label = record['label']

            classify_start = time.time()

            prediction = model.predict(features)[0]

            classify_end = time.time()

            classify_times.append((classify_end - classify_start) * 1000)

            total_times.append((classify_end - arrival_time) * 1000)

            predictions.append(int(prediction))

            actuals.append(int(actual_label))

```



```

        flow_count += 1

except StopIteration:

    pass

session_time = time.time() - start_session

conn.close()

server_socket.close()

# Exclude warm-up flows and report statistics

classify_arr = np.array(classify_times[WARMUP:])

total_arr = np.array(total_times[WARMUP:])

correct = sum(1 for p, a in zip(predictions, actuals) if p == a)

accuracy = correct / flow_count if flow_count > 0 else 0

np.save(RESULTS_PATH + 'classify_times.npy', classify_arr)

np.save(RESULTS_PATH + 'total_times.npy', total_arr)

```

D.5 Traffic Generator — VM1 (`08\_traffic\_generator.py`)

Runs on VM1. Reads flow records from the test set and streams them individually to VM2 over a TCP socket, simulating a live traffic stream.

```

import socket

import json

import time

import pandas as pd

```

```

DATA_PATH = '/home/mathias/ml-ids-project/data/'

```

```

TEST_FILE = DATA_PATH + 'test_selected.csv'

VM2_IP = '192.168.56.102'

PORT = 9999

NUM_FLOWS = 1000

test_df = pd.read_csv(TEST_FILE)

sample = test_df.sample(n=NUM_FLOWS,
random_state=42).reset_index(drop=True)

X_sample = sample.drop('label', axis=1)

y_sample = sample['label']

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client_socket.connect((VM2_IP, PORT))

for i in range(len(X_sample)):

    record = {

        'features': X_sample.iloc[i].tolist(),

        'label': int(y_sample.iloc[i])

    }

    client_socket.sendall((json.dumps(record) + "\n").encode('utf-8'))

client_socket.sendall("END\n".encode('utf-8'))

client_socket.close()

```

D.6 Latency Visualisation (`09\_latency\_chart.py`)

Runs on VM2. Generates the histogram of detector-side processing latency from the saved measurements.

```
import matplotlib

matplotlib.use('Agg')

import matplotlib.pyplot as plt

import numpy as np

RESULTS_PATH = '/home/mathias/ml-ids-project/results/'

total = np.load(RESULTS_PATH + 'total_times.npy')

plt.figure(figsize=(10, 6))

plt.hist(total, bins=40, color='steelblue', edgecolor='black', alpha=0.8)

plt.axvline(np.median(total), color='red', linestyle='--', linewidth=2,
            label='Median: ' + format(np.median(total), '.1f') + ' ms')

plt.axvline(np.mean(total), color='orange', linestyle='--', linewidth=2,
            label='Mean: ' + format(np.mean(total), '.1f') + ' ms')

plt.xlabel('Detector-Side Processing Latency (ms)')

plt.ylabel('Number of Flows')

plt.title('Two-VM Real-Time Detection Latency Distribution (Random Forest, 990 flows)')

plt.legend()

plt.tight_layout()

plt.savefig(RESULTS_PATH + 'realtime_latency_histogram.png', dpi=100,
            bbox_inches='tight')
```